FAAV Build Manual

MIT AURA Lab — 2026 v4.0

MIT AURA Lab

Contents

1 FAAV Build Manual . . . 7
1.1 How this manual is organised . . . 7
1.2 Semester map . . . 7
1.3 Conventions . . . 8
1.4 Architecture in one paragraph . . . 8
1.5 Versioning . . . 8

I Build modules

2 Module 1 — Orientation . . . 11
2.1 The robot . . . 11
2.2 Electronic architecture . . . 11

3 Module 2 — Motor and transmission shaft . . . 13
3.1 What you build today . . . 13
3.2 Before you arrive . . . 13
3.3 The motor — AS2303 KV1500 . . . 13
3.4 Motor shaft . . . 14
3.5 Sensor magnet . . . 15
3.6 Mounting the motor . . . 16

4 Module 3 — Motor driver, sensor and serial . . . 17
4.1 What you build today . . . 17
4.2 Before you arrive . . . 18
4.3 Electronic Speed Controller . . . 18
4.4 Angular sensor . . . 18
4.5 Solder the motor wires . . . 20
4.6 How to program the ESC . . . 20
4.7 Firmware . . . 20
4.8 Commanding and monitoring . . . 20

5 Module 4 — Closed-loop control and tuning . . . 23
5.1 What you build today . . . 23
5.2 Before you arrive . . . 23
5.3 Reduction . . . 23
5.4 Onboard microcontroller . . . 24
5.5 Task sequence . . . 25

5.6 Why low speed is the hard part . . . 25
5.7 Going beyond 600 rad/s . . . 25

6 Module 5 — Frame, crank, linkage and gears . . . 27
6.1 What you build today . . . 27
6.2 Before you arrive . . . 27
6.3 Main body . . . 27
6.4 Crank and linkage . . . 27

7 Module 6 — Flapping simulation (Ptera) . . . 31
7.1 What you do today . . . 31
7.2 Before you arrive . . . 31
7.3 Deliverable . . . 31

8 Module 7 — Wing fabrication . . . 33
8.1 What you build today . . . 33
8.2 Before you arrive . . . 33
8.3 Wing roots . . . 33
8.4 Building the wing . . . 33
8.5 Assembly . . . 34

9 Module 8 — Tail and housing . . . 37
9.1 What you build today . . . 37
9.2 Before you arrive . . . 37
9.3 Tail . . . 37
9.4 Trailing edge holder . . . 37
9.5 Housing . . . 38

10 Module 9 — Flight controller, PCB and dry testing . . . 39
10.1 What you build today . . . 39
10.2 Before you arrive . . . 39
10.3 Electronic architecture . . . 39
10.4 Flight controller . . . 39
10.5 Assembling the PCB . . . 39
10.6 Taking the PCB into operation . . . 40
10.7 Connectors . . . 41
10.8 Dry testing . . . 42

11 Module 10 — Waterproofing . . . 43
11.1 What you build today . . . 43
11.2 Before you arrive . . . 43
11.3 PCB waterproofing . . . 43
11.4 Gearbox . . . 43

12 Module 11 — Wet testing . . . 47
12.1 What you do today . . . 47
12.2 Before you arrive . . . 47
12.3 Pre-immersion checks . . . 47

13 Module 12 — Competition . . . 49
13.1 What happens today . . . 49
13.2 Before you arrive . . . 49
13.3 Feedback . . . 49

II Reference

14 Parts list . . . 53

15 Brushless motors . . . 57
15.1 Start with the brushed DC motor . . . 57
15.2 What changes with a brushless motor . . . 58
15.3 Thermal limits . . . 58
15.4 Measured behaviour of the AS2303 . . . 59
15.5 Why an outrunner . . . 59

16 Software . . . 61
16.1 ESC . . . 61
16.2 Flight controller . . . 61
16.3 PCB . . . 61
16.4 External documentation . . . 61

17 PCB revisions . . . 63
17.1 v3.0 overview . . . 63
17.2 v1.0 — EPFL . . . 64
17.3 v2.0 — MIT . . . 66

18 CNC — Stepcraft D660 . . . 69
18.1 Full guide . . . 69
18.2 Cutpath generation . . . 69
18.3 Material-specific notes . . . 70

19 Design notes . . . 71
19.1 Why an FOC ESC . . . 71
19.2 Speed ceiling and sensor lag . . . 71
19.3 Gearbox corrosion . . . 71
19.4 Housing material tradeoff . . . 71

20 Changelog . . . 73

20.1 fall-2026 . . . 73

3.1 Stock bell . . . 16
3.2 Bring diameter down to 6 mm . . . 16
3.3 Bring the length down to 2.5 mm . . . 16
5.1 Motor shaft . . . 24
5.2 Faulhaber gearbox . . . 24
6.1 Transmission assembled . . . 29
6.2 Transmission, second view . . . 29
8.1 Strips laid out and bonded over the template . . . 34
8.2 Trimming the membrane to the outline . . . 34
10.1 Tape blank PCBs around the PCB to assemble . . . 40
10.2 Align the stencil on top and tape it, making sure it sits flat . . . 40
10.3 Wipe down solder paste with a blade in one firm pass . . . 40
10.4 Place the assembled PCB on the hot plate . . . 40
11.1 Gearbox waterproofing, part 1 . . . 45
11.2 Gearbox waterproofing, part 2 . . . 45
17.1 v1.0 . . . 63
17.2 v2.0 . . . 63
17.3 v3.0 . . . 63

1. FAAV Build Manual

MIT AURA Lab — 2026 v4.0

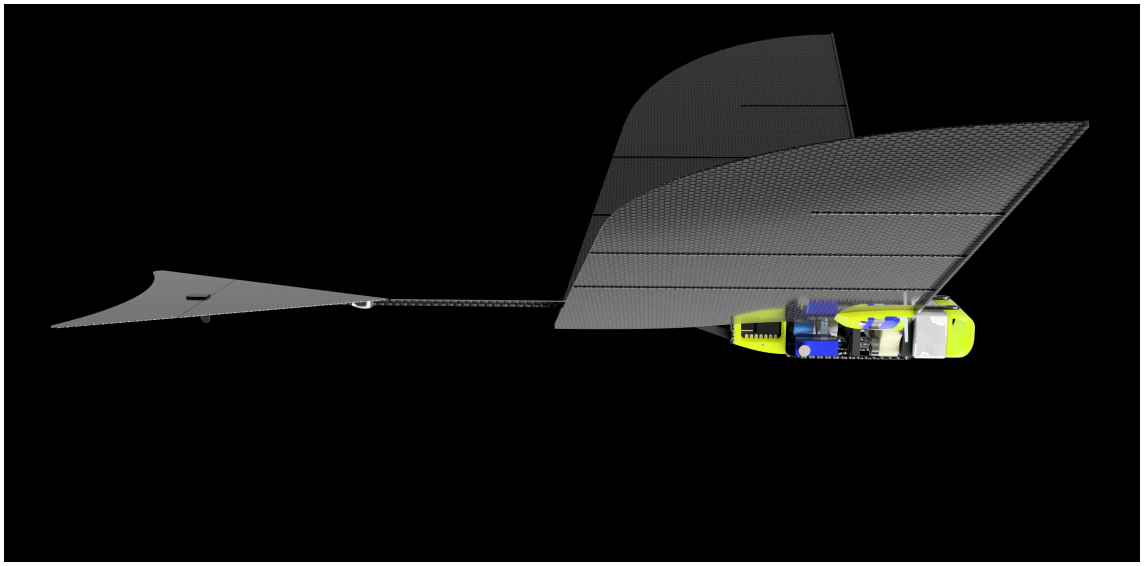


Figure 1.1: The FAAV3.2

Over the semester your team builds a **flapping aerial-aquatic vehicle**: a robot that flies, lands on water, swims by flapping the same wings, and takes off again. This manual is the build side of the course. It is sequenced to the weekly modules, and each page covers exactly what you do in that module.

1.1 How this manual is organised

Build modules are cumulative and in order. Each one opens with what you are building that day, what to have ready before you arrive, and ends with a checkpoint a TA signs off.

Reference holds the material you consult rather than follow: the parts list, the firmware repositories, the PCB revisions, the CNC guide, and the design notes explaining why the robot is the way it is.

1.2 Semester map

Module	Week	Build	Paired lecture
1	Sep 9	Orientation, teams, the robot	Intro to robotic locomotion
2	Sep 14	Motor and transmission shaft	Reaction forces in fluids
3	Sep 21	Motor driver, sensor, serial	Bird-scale flapping design
4	Sep 28	Closed-loop control and tuning	Swimming robots I

Module	Week	Build	Paired lecture
5	Oct 5	Frame, crank, linkage, gears	Undulatory locomotion
—	Oct 12	*No module*	—
6	Oct 19	Flapping simulation (Ptera)	Insect-scale flapping
7	Oct 26	Wing fabrication	Swimming robots II
8	Nov 2	Tail and housing	Multi-modal locomotion
9	Nov 9	Flight controller, PCB, dry testing	The future of actuation
10	Nov 16	Waterproofing	Multirotors
11	Nov 23	Wet testing	Fixed-wing flyers
12	Nov 30	Competition	Pair presentations

1.3 Conventions

> ⚠ Warning
>
> An operation whose likely failure mode destroys a part or injures you. Read it twice before acting.

> ℹ Checkpoint — TA sign-off
>
> Do not proceed past this without a TA looking at your work.

> Why
>
> Design rationale. Skippable at the bench, useful for your report.

1.4 Architecture in one paragraph

One 4S battery (16.8 V max) feeds a custom PCB that steps VBAT down to 5 V to power the flight controller, which in turn generates a 3v3 rail for the sensors on that PCB. Battery power passes through a shunt for current sensing, then out to the ESC. The flight controller commands the ESC over serial. The ESC drives a brushless motor through a 23.04:1 gearbox, and a crank–linkage converts that rotation into wing flapping.

1.5 Versioning

This is version **fall-2026**. Corrections published during the semester are dated in the Changelog. If a step here contradicts what a TA tells you at the bench, the TA is right and this page is stale — report it so the next team does not lose time to it.

I Build modules

tuning 23
5.1 What you build today 23
5.2 Before you arrive 23
5.3 Reduction 23
5.4 Onboard microcontroller 24
5.5 Task sequence 25
5.6 Why low speed is the hard part 25
5.7 Going beyond 600 rpm 25

6 Module 5 — Frame, crank, linkage and gears 27
6.1 What you build today 27
6.2 Before you arrive 27
6.3 Main body 27
6.4 Crank and linkage 27

7 Module 6 — Flapping simulation (Ptera) 31
7.1 What you do today 31
7.2 Before you arrive 31
7.3 Deliverable 31

8 Module 7 — Wing fabrication 33
8.1 What you build today 33
8.2 Before you arrive 33
8.3 Wing roots 33
8.4 Building the wing 33
8.5 Assembly 34

9 Module 8 — Tail and housing 37
9.1 What you build today 37
9.2 Before you arrive 37
9.3 Tail 37
9.4 Trailing edge holder 37
9.5 Housing 38

10 Module 9 — Flight controller, PCB and dry testing 39
10.1 What you build today 39
10.2 Before you arrive 39
10.3 Electronic architecture 39
10.4 Flight controller 39
10.5 Assembling the PCB 39
10.6 Taking the PCB into operation 40
10.7 Connectors 41
10.8 Dry testing 42

11 Module 10 — Waterproofing 43
11.1 What you build today 43
11.2 Before you arrive 43
11.3 PCB waterproofing 43
11.4 Gearbox 43

12 Module 11 — Wet testing 47
12.1 What you do today 47
12.2 Before you arrive 47
12.3 Pre-immersion checks 47

13 Module 12 — Competition 49
13.1 What happens today 49
13.2 Before you arrive 49
13.3 Feedback 49

2. Module 1 — Orientation

Week of Sep 9 · Paired lecture: *Intro to robotic locomotion* · Teams formed today

2.1 The robot

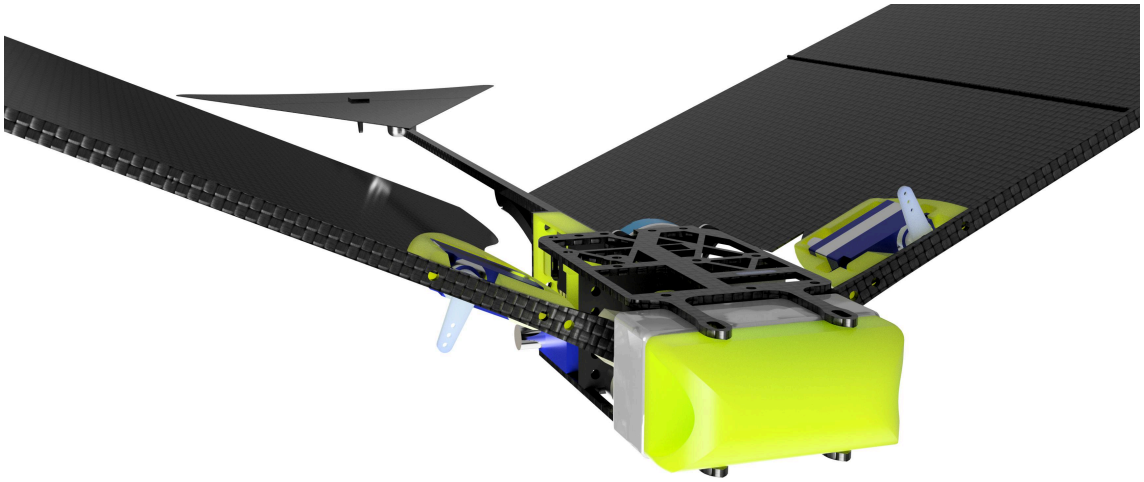


Figure 2.1: The FAAV3.2

The FAAV is a flapping-wing vehicle that operates in air and in water with the same actuation. The build divides into four subsystems, and the modules follow that order:

1. **Transmission** — motor, custom shaft, gearbox, crank and linkage (Modules 2, 5)
2. **Electronics** — ESC, angular sensor, flight controller, custom PCB (Modules 3, 4, 9)
3. **Aero/hydro surfaces** — wings, tail, foam housing (Modules 6–8)
4. **Waterproofing and testing** — potting, immersion, tuning (Modules 10–12)

2.2 Electronic architecture

One 4S battery (max 16.8 V) powers the motor electronic speed controller (ESC). The motor controller generates a 3.3V rail for the encoder and a 5V rail for the flight controller. The flight controller sends commands to the ESC over serial.

> ! To be completed by the instructor
>
> Shop safety requirements, machine access and booking procedure, team size and formation rules, and where the parts crib is. These are institution-specific and are not carried over from the manufacturing guide.

> i Checkpoint — TA sign-off
>
> Kit checked against the parts list, and every member has completed shop training.

3. Module 2 — Motor and transmission shaft

Week of Sep 14 · Paired lecture: *Reaction forces in fluids*

3.1 What you build today

A brushless motor with a custom shaft pressed into its rotor bell, with the sun gear and the sensor magnet installed, bolted to its bracket and mounted on the motor plate. The gearbox itself goes on in Module 4.

3.2 Before you arrive

Parts	AS2303 KV1500 motor with pre-machined bell, custom motor shaft, sun gear from the Faulhaber 23.04:1 gearbox, 6 × 2.5 mm diametric magnet, 3D-printed magnet bracket, vertical nylon motor bracket, bottom fuselage plate, mounting screws
Tools	Circlip pliers, manual arbor press, superglue, hex drivers
Read	This page, and the *Motor shaft* section below in full before touching the motor. Brushless motors for the background

3.3 The motor — AS2303 KV1500

The AS2303 is a **brushless outrunner**. Read Brushless motors first.



We use the AS2303 KV1500 motor throughout the course. Its propeller horn **cannot** be separated from the bell, it has to be machined off in a lathe. That work, and the custom shaft, are prepared for you before the session.

3.3.1 What the KV rating means

KV is the motor's *velocity constant*, quoted in rpm per volt. It says how fast the rotor spins per volt applied, **with no load on the shaft**:

$$\omega_0 = k_V \, U \tag{3.1}$$

The constant is not a marketing number — it falls out of the motor's physics. As the rotor turns, its magnets sweep past the windings and induce a voltage that opposes the supply (the *back-EMF*):

$$U_{\text{emf}} = \frac{\omega}{k_V} \tag{3.2}$$

The supply voltage splits between this back-EMF and the ohmic drop across the winding resistance R, so $U = U_{\text{emf}} + R\,I$ and the speed under load becomes

$$\omega = k_V (U - R\,I) \tag{3.3}$$

which is why Equation (3.1) only holds at $I \approx 0$. Draw current and the motor slows down.

The same constant fixes the torque. Electrical power in equals mechanical power out, $U_{\text{emf}} I = \tau\omega$, and substituting Equation (3.2) gives $\tau = I/k_V$ in SI units. Converting k_V from rpm/V to rad/s/V:

$$k_T = \frac{60}{2\pi\, k_V} \approx \frac{9.55}{k_V} \qquad \text{with} \qquad \tau = k_T\, I \tag{3.4}$$

with k_T in N·m/A when k_V is in rpm/V. **Torque and speed constants are inverses of each other.** A high-KV motor spins fast but gives little torque per amp; a low-KV motor of the same size and mass does the opposite. You cannot buy both — you choose where on that trade-off to sit, and then recover the missing torque with a reduction.

3.3.2 What this means for our robot

With $k_V = 1500$ rpm/V and a 4S LiPo at its 14.8 V nominal voltage:

$$\omega_0 = 1500 \times 14.8 = 22\,200 \text{ rpm} \qquad k_T = \frac{9.55}{1500} = 6.4 \text{ mN·m/A} \tag{3.5}$$

6.4 mN·m/A is very little torque, too little to drive the wings directly, which is exactly why the 23.04:1 Faulhaber gearbox is there. It multiplies torque by 23.04 and divides speed by the same factor. One revolution of the gearbox output is one full flapping cycle, so

$$f_{\text{flap}} = \frac{\omega}{60\,r} = \frac{22\,200}{60 \times 23.04} = 16.1 \text{ Hz} \tag{3.6}$$

at no load. Under real aerodynamic and hydrodynamic loading the drive tops out near 15 200 rpm, i.e. about **11 Hz** of flapping, the figure you will see again in Module 4.

3.4 Motor shaft

To pair the motor with the gearbox, the stock shaft of the brushless motor is replaced by a custom one. You will be handed a **machined bell** and a **finished shaft** — both are prepared before the session. What is left to you is the disassembly and the press fit, which is still a delicate operation: the motor's structure is mostly aluminium and quite thin, for weight savings.

1. Remove the circlip on the shaft with the special pliers. **Don't lose it, nor the brass ring.**
2. Separate the rotor from the stator delicately, by hand.
3. Press the stock shaft out of the rotor bell.

> ⚠ Never load the bell radially
>
> Press the shaft out very carefully, making sure to never apply forces radially on the bell. Use the manual press with appropriate support below the bell to avoid deforming the dome shape. **Do not squeeze the bell with a vice, or even by hand.**

4. Press the sun gear onto the new shaft.

> ⚠ Sun gear first — this order cannot be undone
>
> The sun gear goes on **while the shaft is still free**. Once the shaft is pressed into the bell you can no longer support the shaft properly to press the gear on.

5. Press the new shaft into the bell, making sure not to press on the dome-shaped part of the bell without appropriate support underneath. It is a **tight fit** on the bell and a **clearance fit** through the bearings.

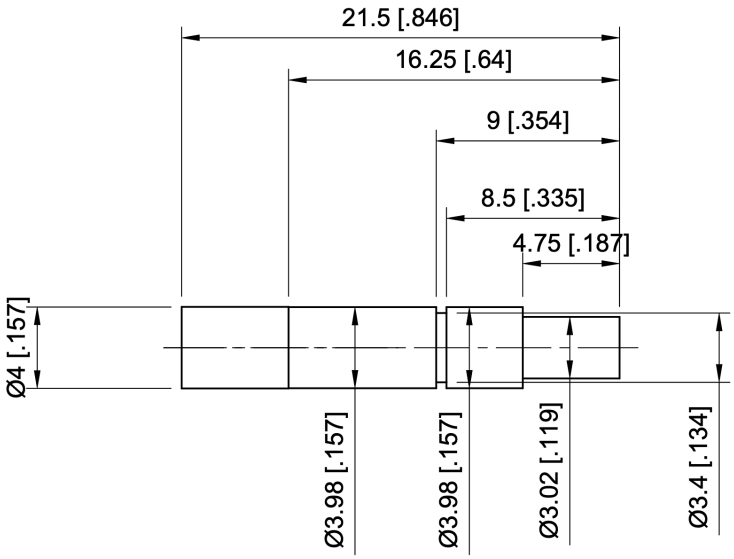


Figure 3.1: Motor shaft dimensions

3.5 Sensor magnet

The AS5047P encoder you wire up in Module 3 does not touch the shaft. It reads the direction of the magnetic field from a small magnet rotating in front of it, so the magnet has to go on the motor.

Use a **6 × 2.5 mm diametrically magnetised disc magnet**. *Diametrically* magnetised is the part that matters: the poles are across the diameter, not through the thickness, so the field seen by the sensor sweeps through a full rotation as the shaft turns. An ordinary axially magnetised disc of identical size presents the same pole to the sensor all the way round and gives no angle at all.

1. Drop the magnet into its recess in the 3D-printed bracket. It should sit flat, fully seated, with no rocking.
2. **Centre it on the axis of rotation.** The bracket locates it, but check. Keep the magnet concentric with the shaft to within **0.5 mm**.
3. Bond it with **a single drop of superglue**. One drop is enough.
4. Turn the assembly through a full revolution by hand and watch the magnet. Any visible wobble means it is off-centre or not seated.

⚠ Off-centre magnets are hard to diagnose later

An eccentric magnet may still produces a perfectly plausible angle reading at times.

3.6 Mounting the motor

With the shaft and the magnet done, the motor goes onto the robot. This happens in two stages: motor to bracket, then bracket to plate.

1. Fix the motor to the vertical **3D-printed nylon bracket** using **four screws** through the mounting holes in the motor base. Don't over tighten. Check that the rotor still turns freely by hand. Nylon is compliant, and overtightening can pull the bracket out of flat enough to preload the bearings. A motor that spun freely on the bench and feels stiff once bolted down is telling you to back the screws off and retighten more gently.
2. Mount the bracket to the **bottom fuselage plate**. Before tightening, check the orientation: the shaft and sun gear must point towards the front of the robot.

! To be completed by the instructor

Fastener specification for both stages — screw size and length for the motor to the bracket, and the count and size for the bracket to the bottom fuselage plate. Length matters most: screws that are too long reach the stator windings inside the motor.

i Checkpoint

Shaft seated with the sun gear on it, the rotor spinning freely by hand with no wobble, the magnet bonded concentric to within 0.5 mm, and the motor bolted to its bracket and mounted on the plate — still turning freely once bolted down.

! To be prepared by the instructor before the session

Both the bells and the shafts must be prepared ahead of time, one set per student. Neither fits in the session: the lathe work is on thin-walled aluminium that is easy to ruin, and there is not enough machine time for the whole cohort.

Machine the bells. Clamp the bell in the lathe using its stock shaft, then take the propeller horn off by bringing the diameter down to **6 mm** and the length to **2.5 mm**.

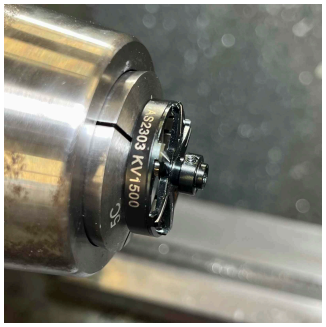


Figure 3.1: Stock bell

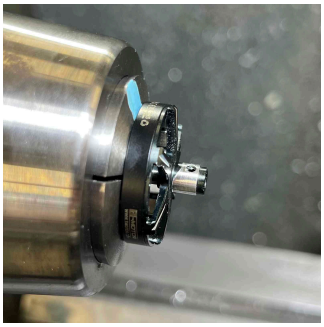


Figure 3.2: Bring diameter down to 6 mm

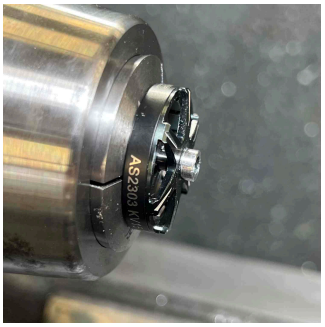


Figure 3.3: Bring the length down to 2.5 mm

Turn the shafts from 4 mm steel rod to the dimensions in Figure 3.1 — clearance fit through the bearings, tight fit on the bell. The 21.5 mm total assumes the motor and gearbox are separated by a **5.5 mm-thick wall**; for a 4 mm wall, shorten the 8.5 mm segment to 7 mm, giving 20 mm overall.

Prepare spares of both — a deformed bell cannot be recovered.

4. Module 3 — Motor driver, sensor and serial

Week of Sep 21 · Paired lecture: *Bird-scale flapping design*

4.1 What you build today

Let's drive our motor! We aim to get a flashed G431 ESC talking to your laptop over micro-USB (serial), with the angular sensor wired on SPI and the motor spinning under commander control.

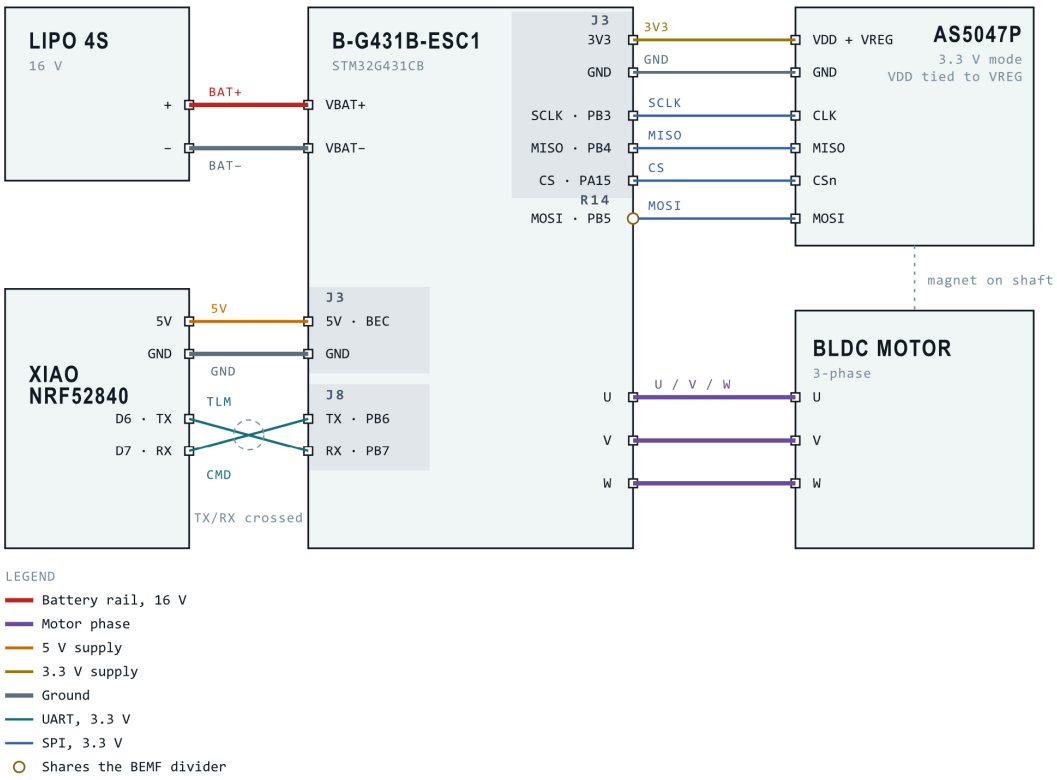


Figure 4.1: How the battery, ESC, encoder, motor and microcontroller connect. The XIAO nRF52840 shown here is the separate-microcontroller option for serial described below — note that TX and RX are crossed. SPI3 and UART pin assignments match the rework in Table 4.1 and *ST-Link serial*.

Work through the module with this diagram open: every step below either wires one of these links or configures it in firmware.

4.2 Before you arrive

Parts needed	B-G431B-ESC1 board, AS5047P encoder, encoder mount, cables
Tools	Soldering iron, tweezers, micro-USB cable, power source
Read	This page, plus Design notes on why we use an FOC ESC

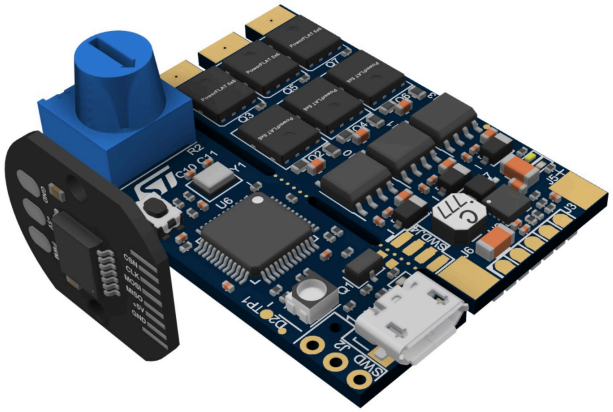


Figure 4.2: ESC running SimpleFOC and the angular sensor

4.3 Electronic Speed Controller

The **B-G431B-ESC1** is ST's reference three-phase motor driver: a complete ESC on one small board, sold as an evaluation kit. Three parts of it matter for this build:

- **An STM32G431 microcontroller.** A 170 MHz Cortex-M4 with hardware floating point. Motor control recomputes the field orientation every control loop. This MCU is the reason we can.
- **A three-phase inverter with current sensing.** Six power MOSFETs drive the three motor phases, and shunt resistors on the low side report the current actually flowing in each. That measurement is what turns "spin the motor" into **torque control**. You cannot regulate a current you cannot see.
- **A detachable ST-Link programmer (the daughterboard).** The small daughter board is how we program the MCU over USB, and it also carries a serial port back to your laptop. When snapped-off, the MCU cannot be reprogramed without physically resoldering it.

It also brings out a hall/encoder connector (**J8**), a CAN transceiver and a potentiometer input, none of which we use as intended. In this build J8 gets repurposed for serial, and the encoder arrives on SPI instead. Both of those reworks are below. The board is rated well above what the wing motor draws, so current is not the constraint here.

4.4 Angular sensor

4.4.1 Why the motor needs an encoder

Field-Oriented Control works by holding the stator's magnetic field roughly 90° ahead of the rotor's magnets, which is the orientation that produces the most torque per amp. To place the field there, the controller has to know where the rotor is **right now**.

Ordinary drone ESCs infer this from back-EMF, i.e the voltage the spinning rotor induces in the phase that is not currently being driven (Equation (3.2) in Module 2). That voltage is proportional to speed, so as the motor slows the signal sinks into the noise and the estimate falls apart. This causes *loss of synchronisation at low speed* described in Design notes. The AS5047P removes the guess. It measures the

field of the magnet bonded to the shaft end and reports the **absolute** angle — valid at standstill just as much as at 15 000 rpm, and correct immediately at power-on with no homing move.

4.4.2 What SPI is, and why we run it at 10 MHz

The AS5047P reports angle over SPI, a four-wire synchronous bus: the controller drives the clock (**SCLK**), data goes out on **MOSI** and comes back on **MISO**, and **CS** selects the chip. Transfer speed is limited only by how fast the two ends can shift bits. We use SPI because it offers **10 MHz** communication. The reason speed matters is that the angle has to be re-read every single control loop, and a stale angle is gad: the controller ends up placing the stator field where the rotor **was**, not where it is. That error shows up as wasted current and audible noise. The same 10 MHz clock is why the wiring has to stay short, see below.

4.4.3 Wiring it to the board

There is no SPI link exposed directly on the ESC board. The solution is to use the **SPI3 bus**, exposed through a few hardware modifications and matching software changes. Figure 4.1 shows the four signals in context, including the MOSI tap on R14.

Table 4.1: SPI3 connections on the G431. * Solder onto the outward side of the resistor.

	SCLK	MISO	MOSI	CS
MCU GPIO	PB3	PB4	PB5	PA15
G431B pin	J3	J3	R14*	J3

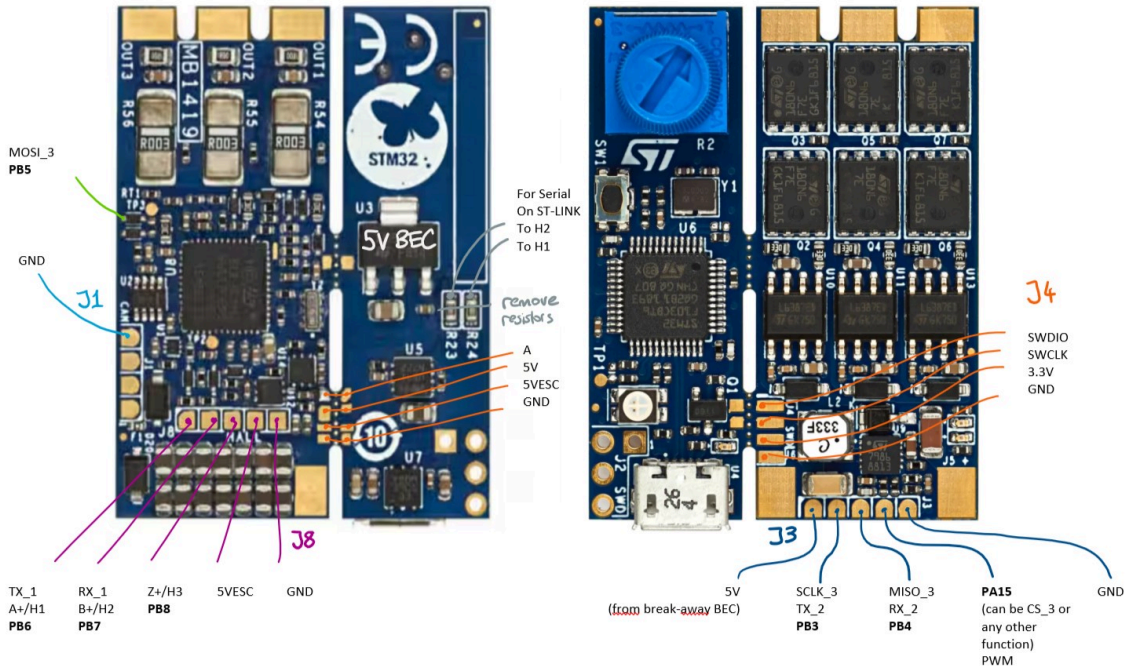


Figure 4.3: GPIOs of the G431B-ESC1 board

> ⚠ Keep the SPI cables under 8 cm
>
> Cables between encoder and ESC need to be **less than 8 cm**. At 10 MHz longer runs start corrupting angle readings, and the symptom is a motor that behaves erratically rather than a sensor that obviously fails.

Next, use M2 screws to mount the encoder on the 3D printed bracket that sits around the motor. Check that the distance between sensor and magnet is 0.5 mm, and centered.

4.5 Solder the motor wires

Since the pads are large, make sure to use enough heat / large tip on the soldering iron. The order of the cables UVW do not matter.

4.6 How to program the ESC

With the encoder and motor connected to the ESC, we can start programing the ESC! The daughterboard is already connected to the ESC MCU. We can program it, but we cannot communicate between our laptop and the ESC. Out of the box that happens via **Serial2**, but we will reuse those GPIOs for the encoder connections. In that case we must connect **Serial1** (on the MCU) to the daughterboard:

1. Disconnect USART2 and connect USART1: remove **R23** and **R24** on the daughter board (desolder).
2. Solder wires to the now-free pins of R23/R24 - the pads closer to U3, further away from the USB port. Connect the wire from **R23** to the pad for hall sensor 2 (**PB7**), and the wire from **R24** to the pad for hall sensor 1 (**PB6**) on J8.
3. Remap the pins in software for SPI use: in `platformio.ini` via `build_flags`, and by overwriting the `WEAK PinMaps` in `pinmap_custom.h`. See commit `18520ac` on GitHub for details.

4.7 Firmware

Code for the B-G431B-ESC1 motor controller, using the SimpleFOC library, is in the lab repository: FAAV-G431-ESC. See Software for all repositories.

4.8 Commanding and monitoring

This is a high-level summary of the SimpleFOC communication docs. Communication with the ESC is over **serial**, either via the ST-Link programmer (the small attached board that we will use for now) or via a separate microcontroller board (that will come next week).

Once serial communication is established, you interact with the ESC through the **commander**, a practical interface for setting and getting values (instructions). Ask the commander for available commands with `?`. Commands available:

Command	Effect
`M`	Full motor configuration
`F`	Get the main loop frequency
`L`	Turn the onboard LED on and off (useful as a comms check)
`T`	Set the target value — rad, rad/s or N/m depending on control mode
`W`	Watchdog on/off. If no new target arrives for 1 s, target is set to zero. **On by default**
`R`	`R0`/`R1` disable/enable a linear ramp when changing setpoint. `RD1000` sets ramp duration to 1000 ms (default 1000 ms). Off by default

> ⚠ Careful when changing control mode
>
> `T` means something different in each mode, position, velocity, or torque. Sending a velocity setpoint while in torque mode will do something you did not intend, with the wings attached. BE VERY CAREFUL ABOUT THIS.

i Checkpoint

Commander responds to `?` over serial at 115200 baud, `L` toggles the onboard LED, the sensor returns a plausible angle through a full manual rotation, and the motor spins under a `T` command.

! To be prepared by the instructor before the session

Prepare a power source 15V 5A over USB-C.

5. Module 4 — Closed-loop control and tuning

Week of Sep 28 · Paired lecture: *Swimming robots I*

5.1 What you build today

The gearbox installed with a marker flag on the output, the Seeeduino wired in and commanding the ESC untethered, running continuously, then holding position, then tracking a velocity setpoint with a tuned loop.

5.2 Before you arrive

Parts	Motor with shaft and sun gear from Module 2, Faulhaber 23.04:1 gearbox, M2.5×10 screws, 3D-printed plates, second vertical bracket, 3× M2.5 and 3× M2 screws, ESC from Module 3, 4x8 mm flanged bearing, Seeeduino XIAO nRF52840, its 3D-printed bracket, 3 wires
Tools	Bench supply with current limit, tachometer or camera, soldering iron
Read	This page in full - the speed limits below explain most of what you will hit

5.3 Reduction

The motor is connected via a custom shaft to a gearbox with a **23.04:1** reduction by Faulhaber.[1] Both parts are held in place by 3D-printed plates. The output shaft of the gearbox has its own bearing on the inside of the gearbox, and is also supported mid-way on the shaft by a 4×8 mm flanged bearing. This leaves **5 mm of shaft length free** for the crank to be mounted on. A flat on the shaft helps lock the rotation of the M3 screw extending through the crank.

[1] Faulhaber gearhead reduction ratios (accessed 20.02.2024).

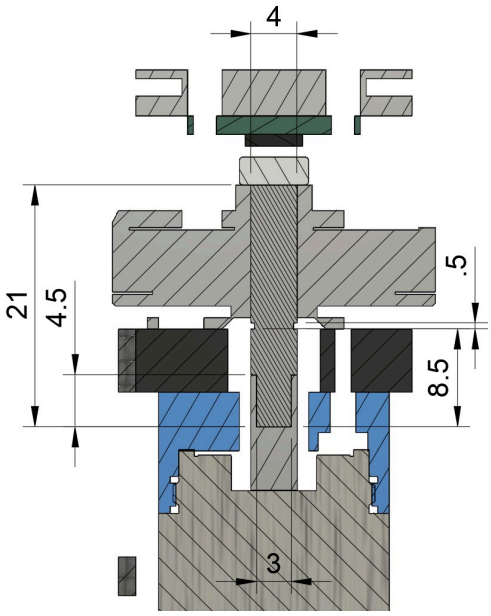


Figure 5.1: Motor shaft



Figure 5.2: Faulhaber gearbox

5.3.1 Installing the reduction

Our sun gear is already pressed onto the motor shaft from Module 2. What is left is mating the gearbox to the motor and fixing the pair to the plates.

1. Slide the gearbox onto the motor shaft so the sun gear engages the planets. It should seat with light hand pressure, if it resists, back off and check the sun gear is square on the shaft rather than forcing it.
2. Fix the blue part of the gearbox: the custom screws that come with the gearbox on one side, and **M2.5×10** screws on the other.
3. Check the output turns freely by hand through a full revolution before applying power.
4. Close the gearbox: the **silver part threads onto the blue part**. Don't tighten it.
5. Attach the **second vertical bracket** to the silver part of the gearbox with **3 × M2.5** screws.
6. Secure that bracket to the **bottom fuselage plate** with **3 × M2** screws. The motor bracket from Module 2 is already on the same plate, so between them the two brackets set the alignment of the whole drivetrain. Check the output still turns freely once both are bolted down.

5.4 Onboard microcontroller

So far every command has come from your laptop over micro-USB. That works on the bench and nowhere else, a robot that has to move cannot stay tethered. The commander interface does not change; what changes is who talks to it. From here on the **Seeeduino XIAO nRF52840** rides on the robot and sends those same commands, and your laptop talks to the Seeeduino instead of to the ESC.

1. Solder the Seeeduino and the ESC's **Serial1**: TX, RX and **GND** lines. In Serial communication, TX and RX are **crossed**, each board's transmit line goes to the other's receive line, as drawn in the wiring diagram in Module 3. GND is not optional: without a shared reference the two boards have no agreed voltage for what counts as a high or low bit. To do this step, you will need to desolder the existing Serial1 TX RX lines to the daughterboard.
2. Install the Seeeduino in its 3D-printed bracket and into the robot.
3. Load the barebone Arduino sketch onto the Seeeduino — see Software for the repository.
4. Modify the sketch to spin the motor at **100 rad/s** and confirm it on the flag you mounted on the gearbox output.

⚠ You lose the micro-USB serial link

Serial1 now goes to the Seeeduino rather than to the daughterboard, so the serial console you used in Module 3 stops working. Flashing the ESC still works — programming runs over SWD, on different pins — but every command and every value you read back now has to pass through the Seeeduino.

Plan for it: anything you still want to inspect over micro-USB, inspect **before** you solder these three wires.

! To be completed by the instructor

Which repository holds the barebone Seeeduino sketch, and on which branch. The nRF52 repositories currently in Software are marked outdated, so students following this page have nothing to clone.

5.5 Task sequence

1. Mount a small flag on the gearbox output so rotation is visible and countable.
2. Achieve continuous spinning at low speed.
3. Achieve position control - the output holds against a hand disturbance.
4. Tune the loop and characterise your maximum usable speed.

! To be completed by the instructor

The specific PID tuning exercise, the target performance, and the sign-off criteria are course material and are not in the manufacturing guide.

5.6 Why low speed is the hard part

Regular drone ESCs start **losing synchronisation at low speed**, which is exactly the regime used for water propulsion. That is the reason this course uses an FOC ESC rather than a hobby unit — see Design notes.

5.7 Going beyond 600 rad/s

Two issues limit speed: the micro-controller loop rate, and delays from the rotation sensor. Using SPI at 10 MHz and an AS5047P sensor rated for 28kRPM speeds are the required hardware changes that improves on delay. But at very high speeds, sensor lag still becomes the binding constraint, and shows up as high current draw and noise. We need to update the sensor electrical angle offset as a function of speed.

5.7.1 Sensor lag

Compensating sensor lag as speed increases is necessary to go above 600 rad/s without wasting current. It is currently hand-tuned every 100 rad/s, and a fitted function is used on the robot. This could be automated by ramping through the speeds and minimising phase current — future work. This SimpleFOC forum thread is useful on the topic.

i Checkpoint

Continuous rotation resulting in frequencies of 1 Hz all the way to 8 Hz without loss of synchronisation, position hold against a hand disturbance, and a recorded loop frequency from F. Do not exceed 8 Hz.

5.7.2 MCU speed

The goal is to maximise main loop frequency, which reached **13 kHz**. Read the current loop frequency with the commander's `F` command.

Change	Effect
`motor.motion_downsample = 3`	+0.8 kHz loop frequency
Disable monitoring	+0.2 kHz
Switch to SVPWM	No improvement observed. May help, unconfirmed
`L = 0.0001`	Tuning the inductance value helped — made voltage mode work much better

6. Module 5 — Frame, crank, linkage and gears

Week of Oct 5 · Paired lecture: *Modelling of undulatory locomotion*

6.1 What you build today

We assemble the carbon frame, the CNC-cut crank and linkage, the gears, and the full mechanical integration that turns motor rotation into flapping. Flapping amplitude is fixed by this design to **60 degrees**[2].

6.2 Before you arrive

Parts	5 mm aluminium link and crank, 2 gears, bearings, M2×4 screws
Tools	drill press, metal glue
Read	This page

6.3 Main body

The main body is a carbon sandwich, spaced **24 mm** apart. The plates are CNC-cut from woven 1.5 mm plates. The top and bottom plates are identical, except for the chamfers needed for the M2 screws. The screws holding plate to plate are **M2×4**, screwing into brass inserts.

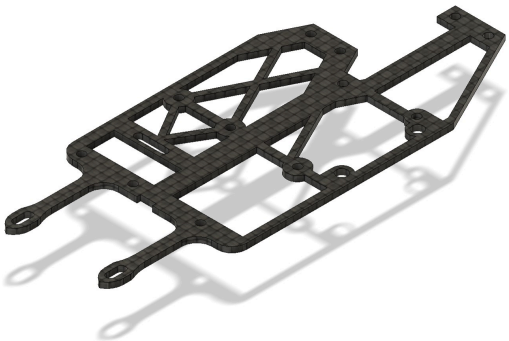


Figure 6.1: Main carbon frame

6.4 Crank and linkage

Dimensions:

[2]Lapsansky AB, Zatz D, Tobalske BW (2020). Alcids 'fly' at efficient Strouhal numbers in both air and water but vary stroke velocity and angle. *eLife* **9**:e55774.

Table 6.1: Crank–linkage dimensions

	Crank-linkage dimensions
Crank radius	4.5 mm
Linkage length	24 mm
Crank–linkage shaft diameter	4 mm
Bearing, crank side	10×4×4
Bearing, wing side	8×3×4 flanged

Both the crank and linkage are CNC-cut in a 5 mm aluminium plate.
Once cut:

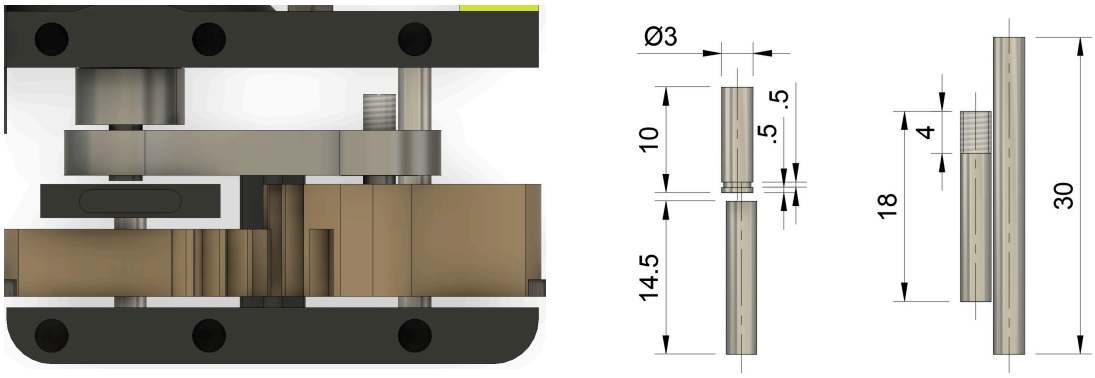
1. Carefully tap an **M3** thread. Use cutting oil and make sure to break off the chips.

> ⚠ This thread carries high forces
>
> It **must** be done carefully, keeping the alignment between hole and tap.

2. Press the bearings into the linkage. They may need to be bonded with metal glue if the press fit is not good enough.
3. Join the crank and linkage with a short 4 mm shaft, cut to **9.5 mm**. The bearing is kept from coming out with a pointy indent on the shaft end. *(To be improved.)*

> ⚠ Alignment of the linkage, perpendicular to the shafts, is extremely important
>
> An offset of **1 mm** is sufficient to destroy the crank-link in a few strokes.

Figure 6.2 shows how the four shafts for the FAAV3.2 should be manufactured. Nothing exotic, but they need to be accurate.



(a) Transmission, top view

(b) Shaft dimensions

Figure 6.2:

The 3 mm shafts are pressed through the two gears' centre holes. Should be a mildly tight fit at minimum, otherwise glue the shaft. Same for the linkage shaft (also 3 mm).

Between the gears, a clearance of **0.4 mm** (inter-axis spacing of 25.4 mm) ensures proper operation without too much backlash.

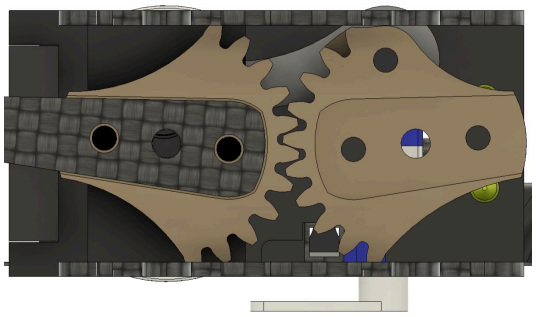


Figure 6.1: Transmission assembled

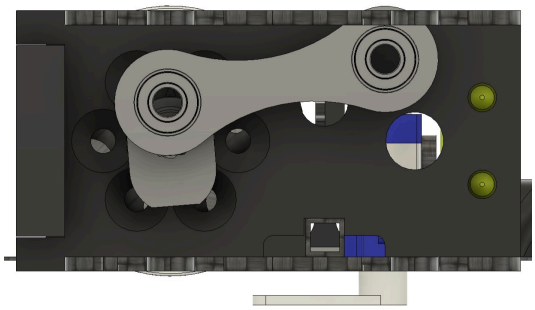


Figure 6.2: Transmission, second view

Checkpoint

Frame assembled, crank–linkage rotating freely through a full cycle with no binding, linkage visibly perpendicular to the shafts, and gear backlash checked. Flapping shown at 1-5 Hz.

7. Module 6 — Flapping simulation (Ptera)

Week of Oct 19 · Paired lecture: *Insect-scale flapping* · No module Oct 12

7.1 What you do today

Choose your wing and tail geometry. You simulate candidate geometries in Ptera and pick the one that gives the best thrust profile when compounded with the provided motor power curve. Modules 7 and 8 build whatever you decide here.

7.2 Before you arrive

Bring	Laptop with Ptera installed, your measured loop performance from Module 4
Read	This page

7.3 Deliverable

A wing planform and a tail geometry, with the simulation output that justifies them, carried forward into Module 7 and Module 8.

! To be written by the instructor

This module has **no counterpart in the manufacturing guide** — the guide documents the research robot's final geometry, not the design exercise. The following need to be written from course material:

- Ptera installation and a worked example
- The provided motor power curve, and its format
- What "most efficient thrust profile" means quantitatively here, and how it is scored
- Constraints the students must respect so their geometry remains buildable with the frame from Module 5 — span limits, root fitting, mass budget
- Whether teams may deviate from the reference wing, and by how much

i Checkpoint — TA sign-off

A geometry chosen, with simulation evidence, and confirmed buildable against the frame and wing-root hardware.

8. Module 7 — Wing fabrication

Week of Oct 26 · Paired lecture: *Swimming robots II*

8.1 What you build today

Both wings: carbon skeleton, membrane cut to the geometry you chose in Module 6, bonded and stitched at the load-bearing points.

8.2 Before you arrive

Parts	Carbon rods and strips, wing roots, 2× nylon adapters per wing, ripstop nylon membrane, 3M tape, superglue, thread
Tools	Cutting mat, steel rule, scalpel, needle, printed full-size template of your geometry
Bring	Your Module 6 geometry, printed 1:1

8.3 Wing roots

The wing roots are carbon, drilled to mate with the gears from Module 5.

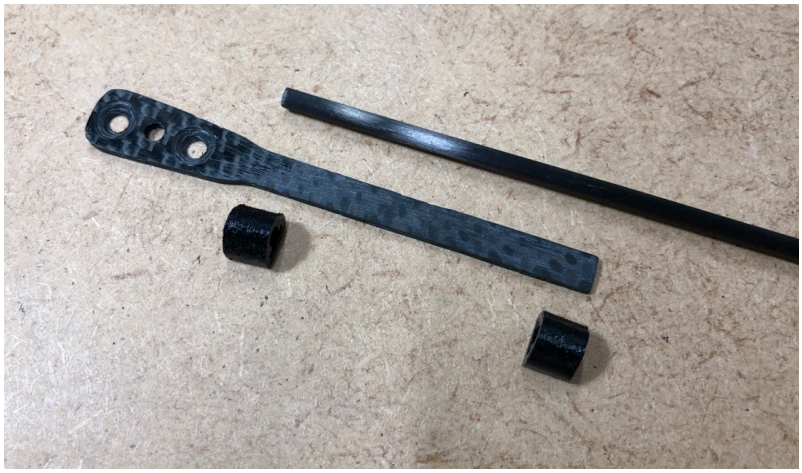


Figure 8.1: Wing root parts and spar stock

8.4 Building the wing

The whole wing is built **over the printed pattern**, working from the leading edge outwards. Everything is cut oversize and trimmed to the outline at the very end — margin is easy to remove and impossible to add back.

1. **Decide on the shape** and print the pattern on paper. Print it **1:1** — check the scale before you cut anything, by measuring a known dimension on the printout against a rule. Printer drivers scale to fit by default, and a pattern that came out at 97% quietly discards the geometry you chose in Module 6.
2. **Cut two membranes** with a **3 cm margin** all round the pattern. Use the ripstop nylon listed in Parts.

3. **Bond the leading edge.** Lay a strip of **3M tape** along the leading-edge region of the membrane, press the leading-edge rod down onto the tape, and wrap the membrane around the rod. This joint carries the highest load of any on the wing — everything downstream pulls against it.
4. **Lay out the carbon strips.** Cut each to approximate length **plus 3 cm**, position them on the membrane over the pattern, and bond them with **superglue**.
5. **Stitch the strips down**, at minimum at every **intersection point** and at the **extremities**. Superglue alone will peel off the membrane under repeated flapping; the stitches are what keep the skeleton attached once the glue starts to fail.
6. **Trim the excess** beyond the outline with a blade, cutting on the mat.

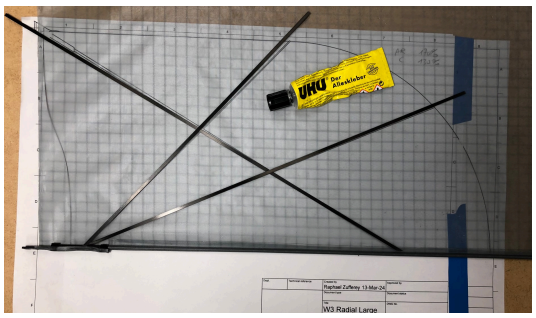


Figure 8.1: Strips laid out and bonded over the template

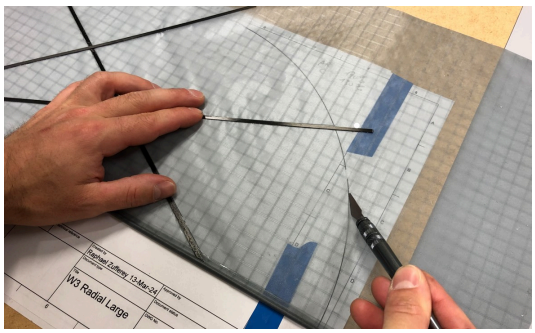


Figure 8.2: Trimming the membrane to the outline

Repeat for the second wing, and remember that it is flipped.

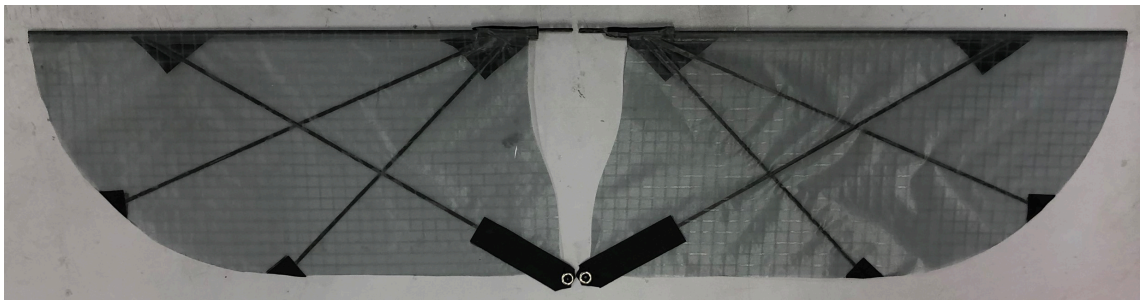


Figure 8.2: Completed wing pair

8.5 Assembly

1. Mount each wing in its **wing root** using the **two nylon adapters**.
2. Mount the wing roots in the robot, onto the gears from Module 5.
3. **Check that it flaps.** Run it slowly first and watch both wings through a full cycle before going any faster.

> ⚠ Both wings must match
>
> Asymmetry between left and right shows up as a roll bias that may not be possible to correct later. See Module 11 for trimming out. Weigh both wings and compare spar positions against the template before bonding.

! To be completed by the instructor

Still missing from the procedure above:

- Which 3M tape — the parts list says VHB, the width and where exactly it lands on the leading edge are not recorded
- Spar layup: rod and strip diameters, and where the diagonals attach
- What to stitch with — thread type and stitch spacing
- Numerical tolerance between left and right wing, for the check below

i Checkpoint

Both wings complete, matched in mass and geometry, mounted in their roots and fitted to the robot, and flapping through a full cycle without binding.

9. Module 8 — Tail and housing

Week of Nov 2 · Paired lecture: *Multi-modal locomotion*

9.1 What you build today

The passive tail, cut to the shape your simulation produced and mounted on its rod, plus the foam housing that protects the robot and keeps it afloat.

9.2 Before you arrive

Parts	Carbon sheet and tail rod, foam stock (XPS or closed-cell PE), epoxy
Tools	Foam cutter, scalpel
Bring	Your Module 6 tail geometry, printed 1:1

9.3 Tail

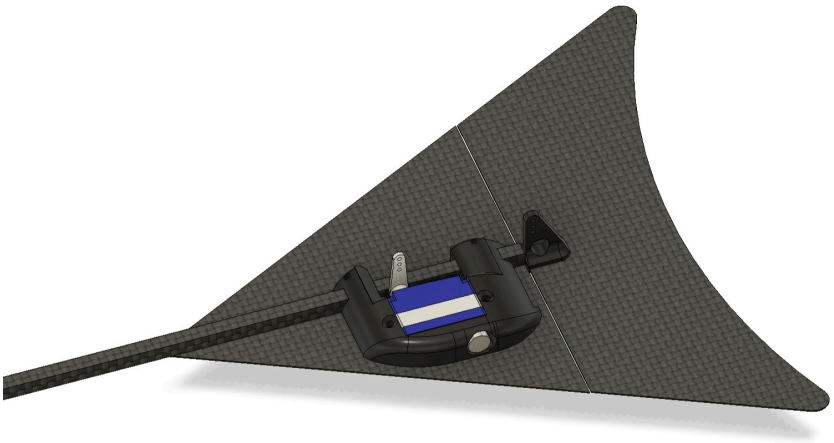


Figure 9.1: Tail assembly

> ! To be written by the instructor
>
> As with the wings, the guide contains **only the photograph above** for the tail. Still needed: sheet material and thickness, rod diameter, how the tail mounts to the boom, and the incidence setting. The project plan also notes tail actuation is optional if time permits — decide before the semester whether that is in scope.

9.4 Trailing edge holder

Insert part on tube.

9.5 Housing

Use a foam cutter to cut an exterior housing for the FAAV. This is an important step for external tests: it adds a layer of protection and ensures the FAAV floats. Leave space for GPS and antenna insertions if you are using them.

To keep the FAAV level when floating — which helps visibility and getting it out of the water — it is recommended to put a certain amount of foam on the tail.

9.5.1 Choosing a foam

Various foams have been tested, with tradeoffs:

Foam	Floatability	Absorption	Impact behaviour
Extruded polystyrene (XPS)	Best, low density	Zero	Relatively brittle; little help damping landings
Closed-cell polyethylene (PE)	Decent, relatively low density	Light	Much more compliant; usable as protection

If someone ever has time, a multi-material housing could have its use cases.

⚠ Check solvent compatibility before painting or gluing foam

Some foams are very sensitive to solvents. Epoxy is inert and can act as glue.

ℹ Checkpoint — TA sign-off

Tail mounted and aligned, housing fitted with clearance for the wings through the full flap cycle, and the assembled robot floating level in the test tank.

10. Module 9 — Flight controller, PCB and dry testing

Week of Nov 9 · Paired lecture: *The future of actuation*

10.1 What you build today

The custom PCB assembled and validated, the flight controller flashed and talking to the ESC, and the robot flapping in air under wireless control.

10.2 Before you arrive

Parts	FAAV PCB v3.0, stencil, solder paste, XIAO ESP32S3, JST connectors, battery
Tools	Hotplate, tweezers, multimeter, bench supply with current limit
Read	This page and PCB revisions

10.3 Electronic architecture

One 4S (max 16.8 V) battery is connected to a custom PCB that steps VBAT down to 5 V to power the flight controller. The flight controller generates a 3v3 rail to power the sensors on the custom PCB. Battery power goes through a shunt for sensing on the custom PCB, then back out to power the ESC. The flight controller sends commands to the ESC over serial.

10.4 Flight controller



Figure 10.1: Microcontroller in its waterproofing box

Firmware lives in FAAV-Flight-Controller (ESP32, FreeRTOS tasks). See Software for older variants.

10.5 Assembling the PCB

As of April 2026 the current board is **v3.0**: 4 layers, 21 × 28.5 mm, 34 components. The complete BOM is in the Manufacturing/Assembly folder of the FAAV-PCB-v3 repository.

The simplest way to assemble a FAAV PCB is with a stencil, solder paste applied with a cutter blade, then reflow on the hotplate.

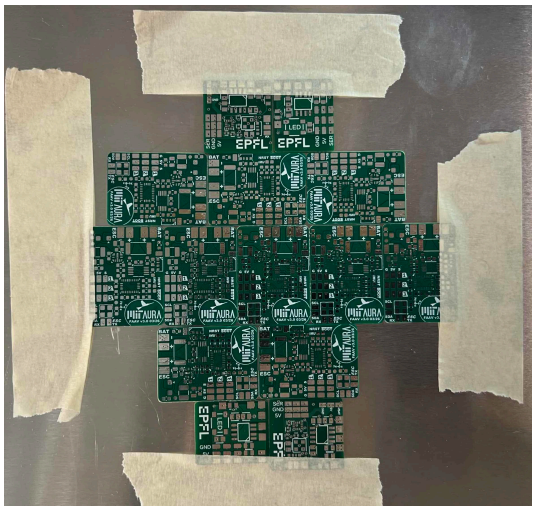


Figure 10.1: Tape blank PCBs around the PCB to assemble

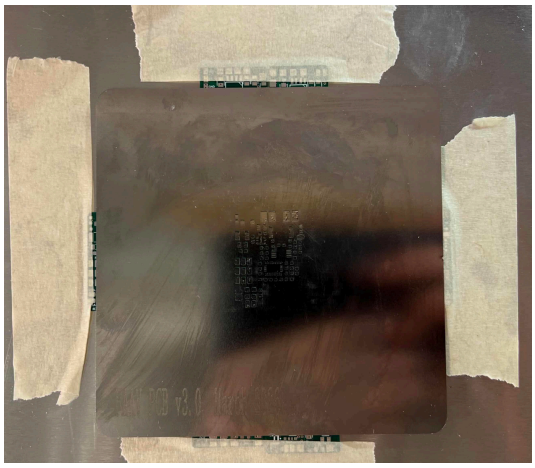


Figure 10.2: Align the stencil on top and tape it, making sure it sits flat

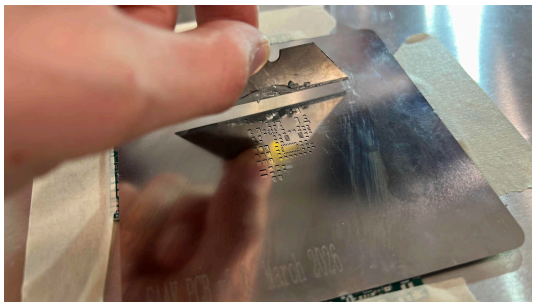


Figure 10.3: Wipe down solder paste with a blade in one firm pass



Figure 10.4: Place the assembled PCB on the hot plate

10.6 Taking the PCB into operation

Once assembled, solder power wires for VBAT and GND at the input side of the board. Follow these steps **before powering the PCB for the first time**:

1. Check that there is no short circuit between GND and 5V / 3.3V / VBAT.
2. Connect a **15 V supply with a 100 mA current limit** to the battery pads and check that it does not draw excessive current.
3. Check with a multimeter that the buck regulator outputs 5 V from the 15 V input. The MCU should be on by then.

With the MCU powered, flash your test firmware and check the connection to all sensors. Table 10.1 gives the I²C addresses.

Table 10.1: I²C addresses

Component	I²C address (bin)	I²C address (hex)
Pressure sensor	`0b1110110`	`0x76`
IMU	`0b1101000`	`0x68`
INA228 power monitor	`0b0101000`	`0x40`

Verify:

- ☐ Voltage, current and temperature readings from the INA228
- ☐ IMU readings
- ☐ Pressure sensor readings
- ☐ RGB LED control

Once all the hardware on the board is validated, solder the power wires at the output side of the board to the ESC, and the UART wires.

> ⚠ TX to RX
>
> The **TX** output of the PCB has to be connected to the **RX** pin of the ESC, and vice versa.

10.7 Connectors

The waterproof connector is the JST 8-pin 08R-JWPF-VSLE-D / 08T-JWPF-VSLE-D. All pins marked _b are used for balancing the battery during recharge. For use on the robot, connect at least **Vbat** and **GND**; to allow higher currents, connect **Vbat_b** and **GND_b** as well.

> ⚠ Seal the unused cable slots
>
> Do not forget to seal the back of the connector for the unused cable slots — this is a water ingress path straight into the electronics.

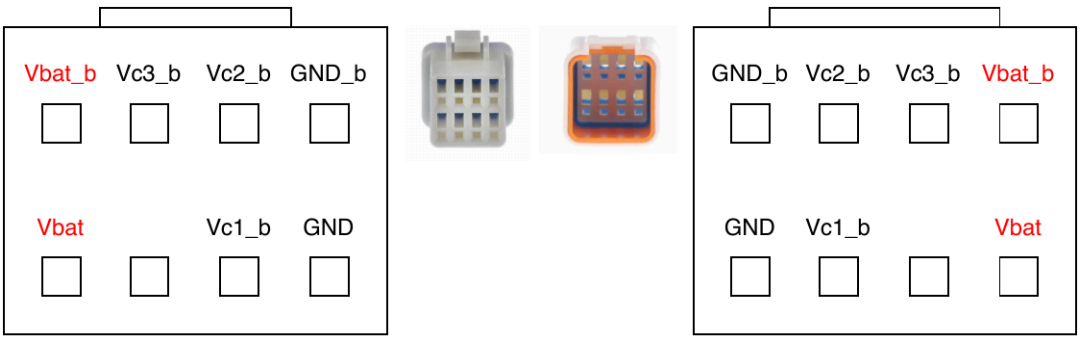


Figure 10.2: Connector wiring scheme. Left: battery connector. Right: connector on the robot / for recharging the battery.

10.8 Dry testing

! To be written by the instructor

The dry-test protocol is course material, not in the guide. Needed: what the robot is mounted on while flapping in air, the flap frequency sweep, what is measured, the wireless control setup students reprogram to, and abort criteria.

i Checkpoint — TA sign-off

PCB validated against all four sensor checks, flight controller commanding the ESC over serial, and a supervised flapping run in air.

11. Module 10 — Waterproofing

Week of Nov 16 · Paired lecture: *Multirotors*

11.1 What you build today

A potted PCB, a lubricated and sealed gearbox, and a robot that survives a static immersion test.

> ⚠ Silicone cure time gates this module
>
> The PCB potting takes **12–24 hours** to cure because of the depth. Plan the module so potting happens first and soldering happens after the cure, or across two sittings.

11.2 Before you arrive

Parts	Waterproofing box, Kapton tape, silicone sealant, USB-C cable, CorrosionX, white lithium grease
Tools	Needle, soldering iron
Read	This whole page before starting — the ordering matters

11.3 PCB waterproofing

First ensure that the assembled PCB fits within the box before anything else.

1. Tape the bottom side of the box with Kapton.
2. Fill the bottom of the box with silicone sealant until it reaches the USB port hole.
3. *Carefully* slide the PCB assembly in, with the Seeeduino facing the bottom of the box.
4. **Very important!** Insert a USB-C cable into the port. This keeps the sealant from flowing into the port, and the cable is a convenient point to hold the assembly by.
5. Add sealant along the three free edges to fill any gaps, until the sealant reaches the level of the top side of the PCB. **The PCB components and contacts should be left exposed.**
6. Let the silicone cure — because of the depth this takes about 12–24 h.
7. Solder the wires and contacts.
8. Solder the voltage regulator.
9. Seal the top side of the PCB. The best method here is unclear. Tested solutions:
   - Nail polish
   - Silicone sealant
   - 422C silicone conformal coating (datasheet)

11.3.1 If you are on a v2.0 board

The method changes slightly. See PCB revisions for the v2.0 procedure, including the SD-card trap-door recovery and antenna potting.

11.4 Gearbox

By default the gearbox on the FAAV4 robot is **not waterproof at all**, and it is normal for it to end up completely filled with water once submerged. Over years of use the gearbox proved largely immune to

corrosion — after more than a year of repeated use in water, including in Lake Geneva, one was opened and showed no sign of corrosion.

That is not the whole story, and the exception is the reason for this step:



Figure 11.1: Gearbox and sun gear after a few dives with motor oil lubrication

Severe corrosion appeared on a newer robot after only 3–4 uses in water. The cause is believed to be improper lubrication after cleaning, combined with the absence of the shaft bearing, which let more water in through the front. Motor oil had been used as a substitute lubricant; its low viscosity is believed to have washed it away quickly, leaving the gears dry. Full investigation in Design notes.

> ⚠ Use the right lubricant
>
> A mix of **CorrosionX and white lithium grease**. Do not substitute motor oil. Several dives have since been performed with the same gearbox that previously corroded, and it now runs smoothly.

Parts that rust: **sun gear, planet gears, ring gear, planet shafts.** Parts that do not: screws, motor shaft, washers, body, circlip.

11.4.1 Sealing modifications

A few optional modifications to the FAAV structure and assembly process keep most of the water out.

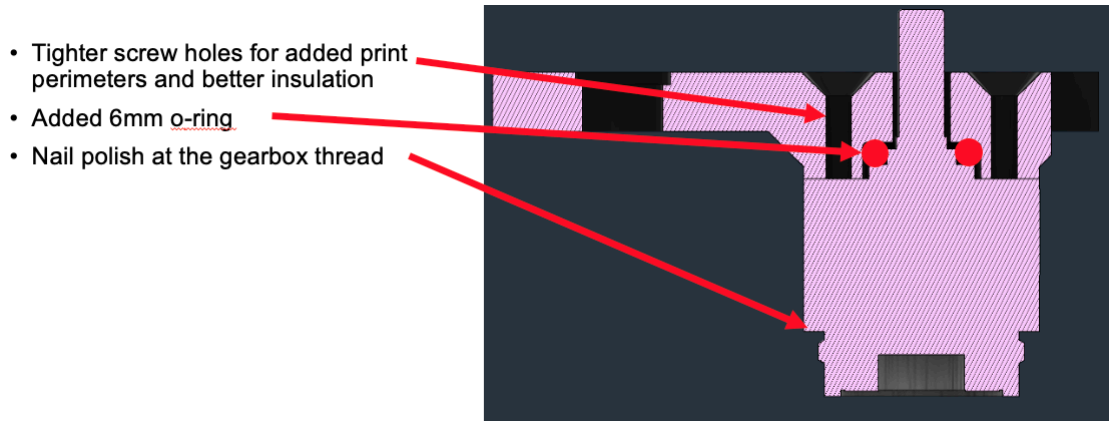


Figure 11.1: Gearbox waterproofing, part 1

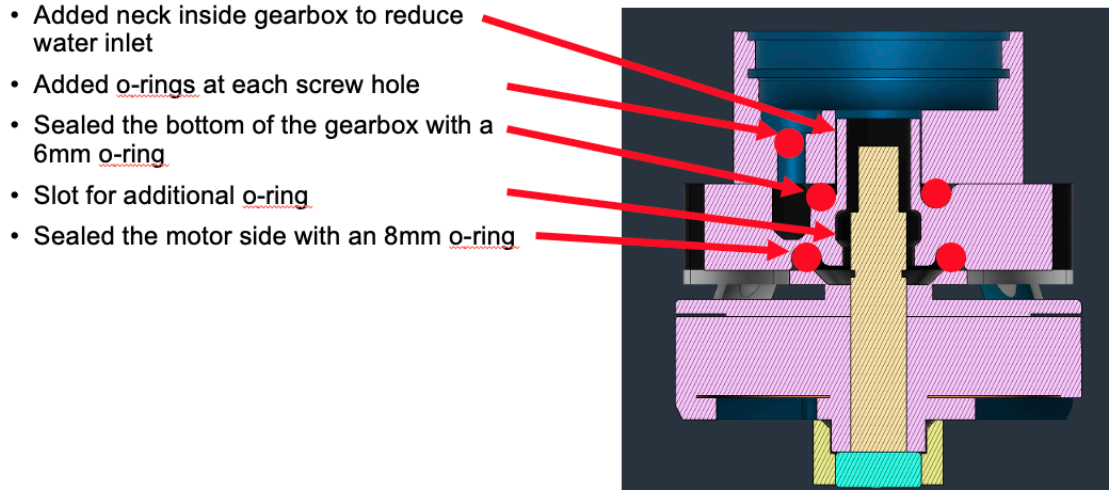


Figure 11.2: Gearbox waterproofing, part 2

> **i** Checkpoint — TA sign-off
>
> PCB fully cured and potted with contacts exposed, gearbox lubricated and reassembled with the shaft bearing fitted, connector back-shell sealed, and a static immersion test passed with no ingress.

12. Module 11 — Wet testing

Week of Nov 23 · Paired lecture: *Fixed-wing flyers*

12.1 What you do today

Immersion, underwater runs, wing trim, and control tuning in water — the regime the whole transmission was designed around.

12.2 Before you arrive

Bring	Fully waterproofed robot from Module 10, charged 4S pack, spare crank–linkage shaft
Read	This page, and your notes from Module 4

12.3 Pre-immersion checks

- ☐ Static immersion test from Module 10 passed, robot opened and inspected dry
- ☐ Connector back-shell sealed on all unused slots
- ☐ Gearbox lubricated with CorrosionX and white lithium grease
- ☐ Watchdog enabled on the ESC (`W`) — target drops to zero if commands stop
- ☐ Robot floats level

> ⚠ Low speed is where hobby ESCs fail
>
> Water propulsion runs the motor in exactly the regime where synchronisation is lost. If the drive stutters underwater, stop and check sensor lag compensation before increasing current — see Module 4.

> ! To be written by the instructor
>
> The wet-test protocol is course material and has no counterpart in the manufacturing guide. Needed:
>
> - Tank or venue, and supervision requirements
> - Run profile: depth, duration, flap frequency sweep
> - What is logged, and how it is retrieved — SD card handling on a potted board is not trivial, see PCB revisions
> - How wing trim is adjusted, and against what reference
> - Cost of transport measurement procedure, since it is scored in Module 12
> - Recovery and abort procedure

> i Checkpoint — TA sign-off
>
> A completed underwater run with logged data, no water ingress on inspection afterwards, and a recorded cost-of-transport measurement.

13. Module 12 — Competition

Week of Nov 30 · Paired lecture: *Pair presentations*

13.1 What happens today

Robots are scored in two categories: **most efficient** and **fastest**.

13.2 Before you arrive

Bring	Robot, spare crank–linkage shaft, spare wing, charged packs, your logged cost-of-transport data

> ! To be written by the instructor
>
> Competition format is course material, not in the manufacturing guide. Needed:
>
> - Course layout and run rules for each category
> - How cost of transport is computed and normalised across teams with different masses
> - Number of attempts, and repair rules between attempts
> - How the competition contributes to the grade
> - What happens to robots afterwards

13.3 Feedback

The final module is a recap and feedback round. This manual is rebuilt each year from what students report — if something in it cost you time, say so. Corrections made during this semester are listed in the Changelog.

II Reference

14 Parts list . . . 53

15 Brushless motors . . . 57
15.1 Start with the brushed DC motor . . . 57
15.2 What changes with a brushless motor . . . 58
15.3 Thermal limits . . . 58
15.4 Measured behaviour of the AS2303 . . . 59
15.5 Why an outrunner . . . 59

16 Software . . . 61
16.1 ESC . . . 61
16.2 Flight controller . . . 61
16.3 PCB . . . 61
16.4 External documentation . . . 61

17 PCB revisions . . . 63
17.1 v3.0 overview . . . 63
17.2 v1.0 — EPFL . . . 64
17.3 v2.0 — MIT . . . 66

18 CNC — Stepcraft D660 . . . 69
18.1 Full guide . . . 69
18.2 Cutpath generation . . . 69
18.3 Material-specific notes . . . 70

19 Design notes . . . 71
19.1 Why an FOC ESC . . . 71
19.2 Speed ceiling and sensor lag . . . 71
19.3 Gearbox corrosion . . . 71
19.4 Housing material tradeoff . . . 71

20 Changelog . . . 73
20.1 fall-2026 . . . 73

14. Parts list

Table 14.1: List of components, after Table S1

Category	Component	Material/ Model	Nb	Mass [g]	Total [g]
Structure	Frame	Carbon 1.5mm plate (swiss-composite)	2	4.7	9.4
	Vertical panels	Onyx with CF (Markforged)	2	6.2	12.4
	Motor holder	Onyx with CF (Markforged)	1	4.3	4.3
	Battery holder	Onyx with CF (Markforged)	1	1.1	1.1
	Tail holder	Onyx with CF (Markforged)	1	4.3	4.3
	Screws	M2 Stainless	40	0.13	5.2
Tail	Tail	Carbon 0.5 mm plate (Swiss-composite)	1	22.2	22.2
Propulsion	Crank	Al 7075	1	1.3	1.3
	Linkage	Al 7075, with bearings	1	4.3	4.3
	Reduction	Series 20/1R 1:23 (Faulhaber)	1	38	38
	Right plate	Carbon 1.5mm plate	1	1.5	1.5
	Left wing gear	Onyx with CF (Markforged)	1	2.8	2.8
	Right wing gear	Onyx with CF (Markforged)	1	6.7	6.7
Electronics	Flight controller	Seeeduino NRF52840	1	2.4	2.4
	Motor	AS2303 1500kV (T-Motors)	1	17	17
	Hall sensor	AS5047P (ams-OS-RAM) at 10 MHz SPI	1	1.2	1.2
Battery proof.	[illegible]	[illegible]	2	[illegible]	[illegible]

! To be completed before the semester

This is the published list for the paper's robot, not the current build. Items the modules rely on that are **not** in it and need adding: the AS2303 motor (the list still shows the older AT2303), the ST-Link programmer, the FAAV PCB v3.0 and its BOM, 4 mm steel rod for the motor shaft, the 6 × 2.5 mm diametric sensor magnet and its printed bracket, 5 mm PEEK stock, foam and CorrosionX.

15. Brushless motors

Background for Module 2. Not needed at the bench — useful for your report, and for understanding why the motor behaves the way it does when you push it in Module 4.

15.1 Start with the brushed DC motor

Every result you need for a brushless motor is easier to see in a brushed one first, because the two obey the same electrical physics. The difference is *how* the windings get switched, not what the machine does once they are.

A DC motor is described by two constants and one resistance. Spinning the rotor induces a back-EMF proportional to speed, and passing current through the windings produces torque proportional to current:

$$U_{\text{emf}} = \frac{\omega}{k_V} \qquad \tau = k_T\, I \tag{15.1}$$

These are the same k_V and k_T derived in Module 2 — inverses of each other, not independent choices. The supply voltage splits between the back-EMF and the ohmic drop across the winding resistance R:

$$U = \frac{\omega}{k_V} + R\, I \tag{15.2}$$

Substitute $I = \tau / k_T$ and rearrange for torque, and the whole behaviour of the motor falls out as a **straight line**:

$$\tau = k_T \frac{U}{R} - \frac{k_T}{k_V R}\,\omega \tag{15.3}$$

15.1.1 Reading the torque–speed line

That line is the motor's entire steady-state character, and it is fixed by the applied voltage. Two points define it:

- **Stall torque** $\tau_{\text{stall}} = k_T U / R$, at $\omega = 0$. The rotor is held still, there is no back-EMF, and the only thing limiting current is the winding resistance. This is the largest torque the motor can produce and also the fastest way to destroy it — see Thermal limits.
- **No-load speed** $\omega_0 = k_V U$, at $\tau = 0$. The rotor spins fast enough that the back-EMF nearly cancels the supply, so almost no current flows.

Everything the motor can do at that voltage lies on the line between them. Mechanical power is $P = \tau\omega$, the product of a falling line and a rising one, so it is a parabola peaking at **half the no-load speed** with half the stall torque. Efficiency peaks somewhere else entirely — much closer to ω_0, because the copper loss that dominates at low speed scales with I^2 while the useful output only scales with I.

(The web version of this page carries an interactive plot of these three curves. See the manual online.)

> **i** The useful consequence
>
> Raising the voltage shifts the line outwards without changing its slope: more speed *and* more torque, same trade-off between them. That is why speed control is voltage control, and why a "faster" motor at fixed voltage is a lower-torque one.

(Online, this section carries a voltage slider showing the line sweeping outwards.)

15.2 What changes with a brushless motor

In a brushed motor the commutator and brushes are a mechanical switch riding on the rotor: they physically reconnect the windings as it turns, so the current always flows in whichever coil is best placed to produce torque. It works without any electronics at all, and it wears out, arcs, and limits how fast the motor can safely spin.

A **BLDC** motor deletes that switch. The permanent magnets go on the rotor, the windings stay on the stator in three phases, and an external controller decides which phases to energise based on where the rotor currently is. The motor becomes inseparable from its driver — this is why Module 3 exists, and why the rotor position measurement matters so much.

What this does **not** change is Equation (15.3). The torque and speed constants have the same meaning, the line has the same shape, and you size the motor the same way. What changes is that the line is no longer the whole story:

	Brushed	Brushless
Commutation	Mechanical, self-acting	Electronic, needs rotor position
Torque–speed line	The real limit	The *ideal* limit, then clipped by the driver
Wear	Brushes are consumable	Bearings only
Speed ceiling	Brush arcing and bounce	Commutation rate and sensor lag
Torque ripple	Smoothed by many commutator segments	Depends on drive scheme (trapezoidal vs FOC)

15.2.1 The envelope, not the line

The straight line assumes the supply can deliver whatever current the equations ask for. It cannot. At stall, Equation (15.2) gives $I = U/R$, and with the sub-ohm winding resistance of a small outrunner that is a number in the hundreds of amps — far beyond both the ESC and the motor itself.

So the driver enforces a **current limit**, and because $\tau = k_T I$ that is directly a torque limit. The usable region becomes:

- a **constant-torque region** at low speed, flat at whatever current limit you set, because the ESC is holding current rather than letting the line dictate it;
- a **corner speed** where the back-EMF has risen enough that the supply can no longer push that current;
- above it, the original Equation (15.3) line, falling to ω_0.

The flat part is not physics, it is a decision. On our robot it is the decision that keeps the motor alive at the low speeds used for swimming, where the torque–speed line alone would happily draw enough current to cook the windings.

15.3 Thermal limits

Nothing in Equation (15.3) knows about heat, which is why the equations cheerfully predict operating points that will destroy the motor in seconds.

Almost all the loss you care about is **copper loss** in the windings:

$$P_{\text{copper}} = I^2 R \tag{15.4}$$

The square is what makes this the binding constraint. Doubling the torque doubles the current and **quadruples** the heating. A motor comfortable at its rated current will not survive twice that for long, even though the torque equation says it is only a factor of two.

Three practical consequences:

1. **Stall is the worst case, not the best.** At $\omega = 0$ there is maximum current and no rotation, so no airflow and no useful output — all the input power becomes heat in the windings. A motor held at stall can burn out without the shaft moving at all.

2. **Continuous and peak ratings are different numbers.** A peak rating assumes seconds; a continuous rating assumes thermal equilibrium. Flapping is a *reciprocating* load, so the current is far from constant — what matters for heating is the **RMS** current over the cycle, not the peak you see on the bench supply.
3. **Iron losses grow with speed.** Hysteresis and eddy-current losses in the stator scale with electrical frequency, so at the 1700 rad/s discussed in Module 4 the copper is no longer the only thing getting warm.

(Online, this section carries sliders for the ESC current limit and the motor's continuous rating, showing how the two clip the torque–speed line.)

> ⚠ Heat is cumulative and invisible
>
> Current is the only symptom you can see, and it appears instantly, while the temperature it causes lags behind by tens of seconds. A run that looks fine on the ammeter can still be depositing heat faster than the motor sheds it. When tuning in Module 4, stop and touch the motor.

15.4 Measured behaviour of the AS2303

Everything above is the model. The plot below is where the **test-stand measurements** go, so you can see how far the straight line actually holds for the motor on your bench.

> ! To be completed by the instructor
>
> Two things, and the second depends on the first:
>
> 1. The measured winding resistance of the AS2303, its continuous current rating, and the current limit configured on the ESC. These turn every equation on this page into numbers students can check against — the plots above currently run on a **placeholder** R.
> 2. The test-stand data itself. Paste it into the `MEASURED` array in the script block at the bottom of `reference/bldc.qmd`; the schema and an example row are in the comment there. The plot switches from its empty state to a measured-versus-model comparison as soon as the array is non-empty.

15.5 Why an outrunner

Brushless motors come in two arrangements. An **inrunner** spins a magnet rotor inside a stationary stator, the familiar shape from a drill. An **outrunner** inverts it: the stator sits still in the middle and the magnets are carried on a bell that rotates around the outside. Ours is an outrunner — it is the bell you machine and press the shaft into in Module 2.

The reason is torque. Torque is force times radius, and in an outrunner the magnets sit at the *largest* radius the motor's diameter allows, rather than the smallest:

$$\tau = F \times r \tag{15.5}$$

The same tangential force from the same windings acts on a longer lever arm. It also leaves room for more magnet poles around the circumference. Together these raise k_T for a motor of the same mass, which by Equation (15.1) means a **lower KV** — more torque per amp, less speed per volt.

That is the right trade for us. The wings need torque at low speed, through a gearbox that is already multiplying by 23.04:1, and the alternative is either a larger reduction or more current for the same result. Two secondary properties help as well:

- **Rotor inertia.** The rotating mass is at a large radius, so the bell has significant inertia and acts as a flywheel. Against a reciprocating flapping load that smooths the speed variation over a cycle. It also means the motor cannot change speed quickly, which is a real cost during transients.
- **Something to attach to.** The bell is a large-diameter rotating surface, which is what makes the shaft press-fit in Module 2 mechanically sensible in the first place.

The costs are real: outrunners spin more slowly for a given voltage, the exposed rotating bell is a hazard and an ingress path, and cooling is worse because the heat-producing stator is wrapped inside the rotor

rather than mounted against the case. The last point ties directly back to Thermal limits — an outrunner's stator sheds heat mainly through its mount, which on our robot is a 3D-printed plate.

16. Software

16.1 ESC

Code for the B-G431B-ESC1 motor controller using the SimpleFOC library: FAAV-G431-ESC.
Used in Module 3 and Module 4.

16.2 Flight controller

- **Current** — ESP32, based on FreeRTOS tasks: FAAV-Flight-Controller

Outdated versions, kept for reference:

- Old EPFL code for NRF52: FAAV-Seeeduino
- Old MIT code for NRF52/ESP32: FAAV-AUK

16.3 PCB

- FAAV-PCB-v3 — current board, including the BOM in the Manufacturing/Assembly folder

16.4 External documentation

- SimpleFOC communication
- SimpleFOC commander
- XIAO ESP32S3 getting started

> ! Access
>
> Confirm before the semester that students have read access to these repositories, and pin the branch or tag each module expects. “Latest main” changing under a cohort mid-semester is the most likely way this manual goes stale without anyone noticing.

17. PCB revisions

To interface the MCU to the rest of the FAAV, a custom PCB is required. As of April 2026 the latest board in the FAAV is **v3.0**. More information is in the FAAV-PCB-v3 repository.

Version	Also called	Notes
v1.0	EPFL PCB	Details below
v2.0	MIT PCB	Details below
v3.0	FAAV PCB v3.0	Current board

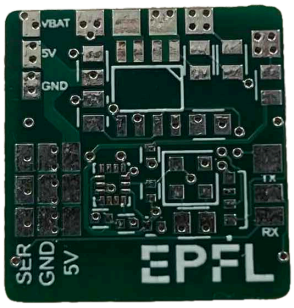


Figure 17.1: v1.0

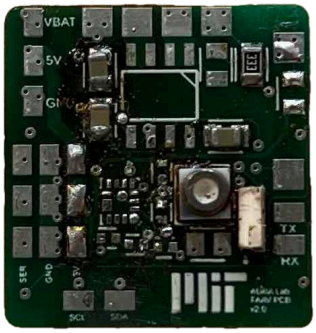


Figure 17.2: v2.0

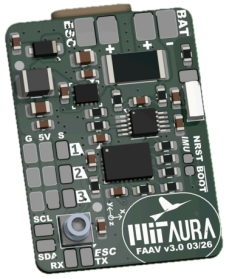


Figure 17.3: v3.0

17.1 v3.0 overview

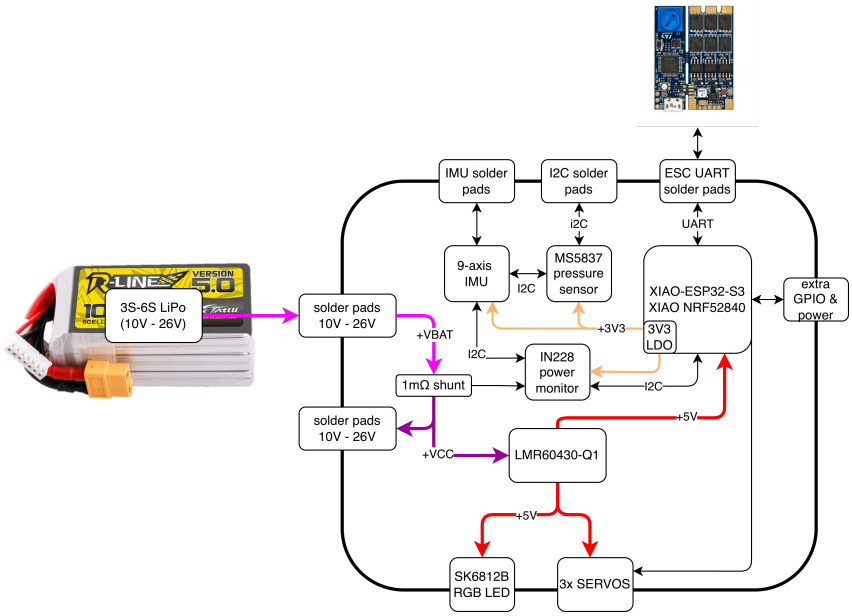


Figure 17.1: FAAV PCB v3.0 block diagram

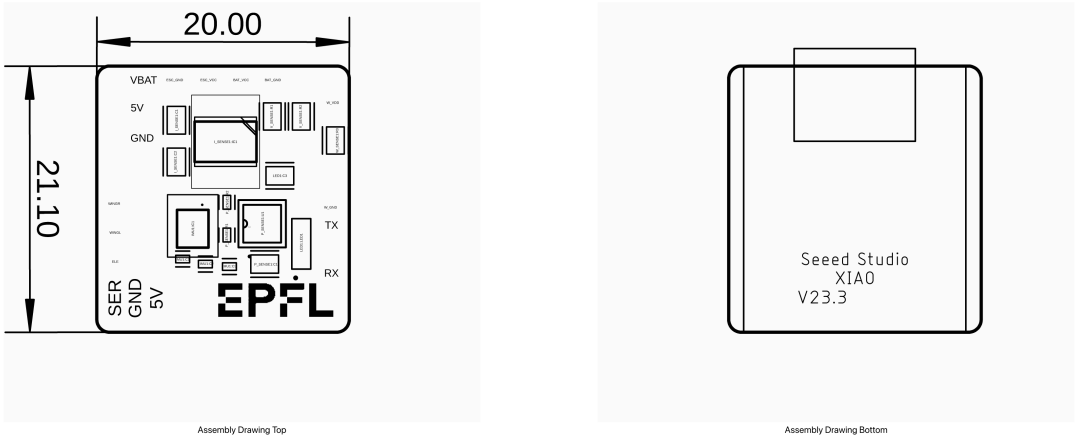
The PCB is 4 layers, 21 × 28.5 mm, and has 34 components. The complete BOM is in the Manufacturing/Assembly folder of the repository.

Assembly and bring-up are covered in Module 9.

17.2 v1.0 — EPFL

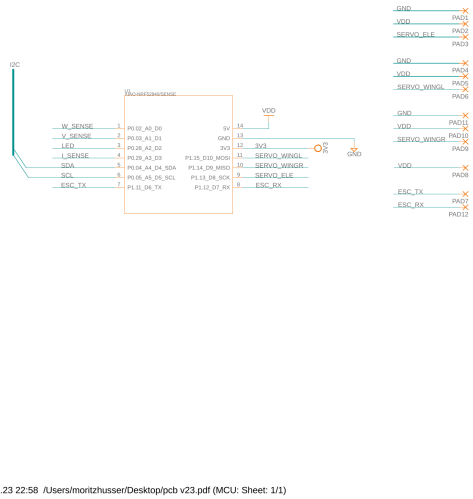
A 2-layer PCB. Its purpose is to connect the flight controller to the battery, ESC and so on, and to provide additional functionality. It was designed with the Fusion360 PCB designer. The PCB contains a current sensor, a voltage sensing circuit, a pressure sensor for measuring water depth, water sensing probes, an LED, an IMU, and several pads for soldering cables. The IMU and current sensor connect to the MCU over I²C.

> ⚠ Kapton between flight controller and PCB
>
> To prevent short circuits, put a layer of Kapton tape between the flight controller and the PCB when soldering the Seeeduino to the PCB.

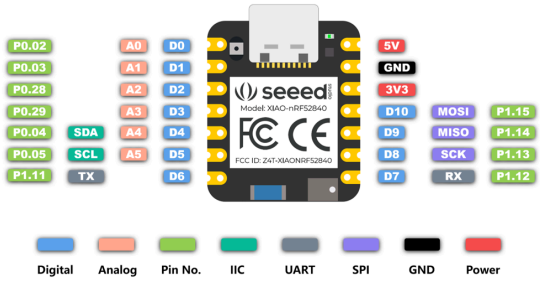


(a) Top (b) Bottom

Figure 17.2: PCB v1.0 assembly drawings



(a) MCU pin mapping on PCB



(b) Pin map for XIAO nRF52840 (Sense) MCU

Figure 17.3: Pin mapping of PCB pins and flight controller pins

17.2.1 Component list

Table 17.1: PCB v1.0 components

Names	Component description	Supplier	Item number
IMU1:C1	Capacitor 0.1 µF (0402)	Mouser	581-KGM05AR51A104KH
I_SENSE1:C1, LED1:C3, P_SENSE1:C1	Capacitor 0.1 µF (0805)		
P_SENSE1:R1, P_SENSE1:R2	Resistor 10 k (0402)	Mouser	71-CRCW040210K0FKEDC
IMU1:C2	Capacitor 10 nF (0402)	Mouser	581-KGM05AR71C103KH
I_SENSE1:C2	Capacitor 10 nF (0805)		
W_SENSE1:R3	Resistor 1 M (0805)		
IMU1:C3	Capacitor 2.2 µF (0402)	Mouser	581-KGM05AR51A225KH
V_SENSE1:R1	Resistor 330 k (0805)		
V_SENSE1:R2	Resistor 33 k (0805)		
LED1:LED1	LED 4691 Adafruit	Mouser	485-4691
I_SENSE1:IC1	Current sensor ACS723LLCTR-20AU-T, Allegro	Mouser	250-723LLCTR20AUT
IMU1:IC1	IMU ICM-42688-P, TDK InvenSense	Mouser	410-ICM-42688-P
P_SENSE1:U1	MS583730BA01-50, TE Connectivity	Mouser	824-MS583730BA01-50
I_SENSE1:BAT_GND_REG, I_SENSE1:BAT_VCC_REG	Pin header		
MCU1:U1	XIAO nRF52840 / Sense, Seeed Studio		

17.3 v2.0 — MIT

The goal of this version was to fix existing functional problems (I²C, pad spacing) and to introduce a new external I²C pad for GPS functionality. It is also designed to function by default with both the newer ESP32S3 chips and NRF52 chips. Improvements in range through WiFi connection are expected. **Manufacturing steps are identical to v1.0.**

17.3.1 ESP32 compatibility

Hardware and software arrangements were needed for XIAO ESP32S3 compatibility. For software see Software.

Hardware changes: a new PCB housing part allowing SD card removal and reinsertion, a WiFi antenna housing case, and a GPS housing case.

SD card notes

1. SD cards should be formatted to a specific format (FAT32, believed).
2. Use non-sharp tweezers to gently extract the SD card from the PCB case — otherwise you risk breaking the card and making it unreadable.
3. For simultaneous servo and SD card usage, be careful not to use the same pins as on the previous chip for servos; TX and RX are used for SD card communication. You may use the secondary microSD card PCB's two additional pins.

17.3.2 Waterproofing

The method changes slightly from Module 10. Admittedly it can be vastly improved, but it works:

1. Insert the assembled PCB, making sure the secondary SD card PCB does not come loose, and that the WiFi antenna is properly inserted.
2. Insert the USB-C cable into the port, and a microSD card in the relevant slit.
3. Tape the bottom of the PCB so no silicone escapes when pouring. Pour silicone, using a needle to ensure there are no air bubbles between layers.
4. When fully cured, the SD card is very hard to remove if the protective section broke off during assembly. It can be salvaged by cutting around three sides of the SD card with a knife and opening it like a trap door, freeing the stuck sections. The trap door can then be sealed with epoxy or more silicone — only on the surface, as you do not want to block the SD card again.

The WiFi antenna should also be potted. Currently the same housing case as for the ESC is used: tape the bottom of the box with Kapton tape, stick the antenna to the bottom of the casing (normally all in sticker form), then pot it with silicone. The silicone layer should be left relatively thin.

> **i** The antenna stops working when wet
>
> It works again once dried. This is expected behaviour, not a fault.

17.3.3 Points for a redesign

- Incorrect wiring for pressure sensor — SDA and SCL are swapped.
- Cover the vias (currently open vias).
- Vias on pads of small SMD components are not optimal for soldering, or just cover the vias.
- Increase pad spacing for the cables.
- Current sensor: possibly change to a 3.3 V part. Currently a 5 V sensor is used, so current should not exceed $(3.3\,\mathrm{V} - 0.5\,\mathrm{V})/0.2\,\mathrm{V/A} = 14\,\mathrm{A}$ without risking damage to the analog ports of the Seeeduino.
- ~~UART RX and TX are swapped~~ *(resolved)*

18. CNC — Stepcraft D660

Used in Module 5 for the carbon frame, the aluminium crank and linkage, and the PEEK gears.

18.1 Full guide

The complete Stepcraft guide is available as a PDF:

- CNC Stepcraft D600 LIS guide (PDF)
- Original wiki version: wiki.epfl.ch/lis/cnc-stepcraft

> ⚠ Before every job
>
> Make sure the **0 of the machine is at the right location in your file**, and check the orientation of the axes.

18.2 Cutpath generation

Fusion360 generates cutpaths from the parts in the *manufacture* tab, and allows for *manufacturing models*, where parts can be recombined, realigned and even modified. Chamfers on the body plates were added in the manufacturing model rather than in the design.

Operation order: **holes → chamfers → pockets → outline.**

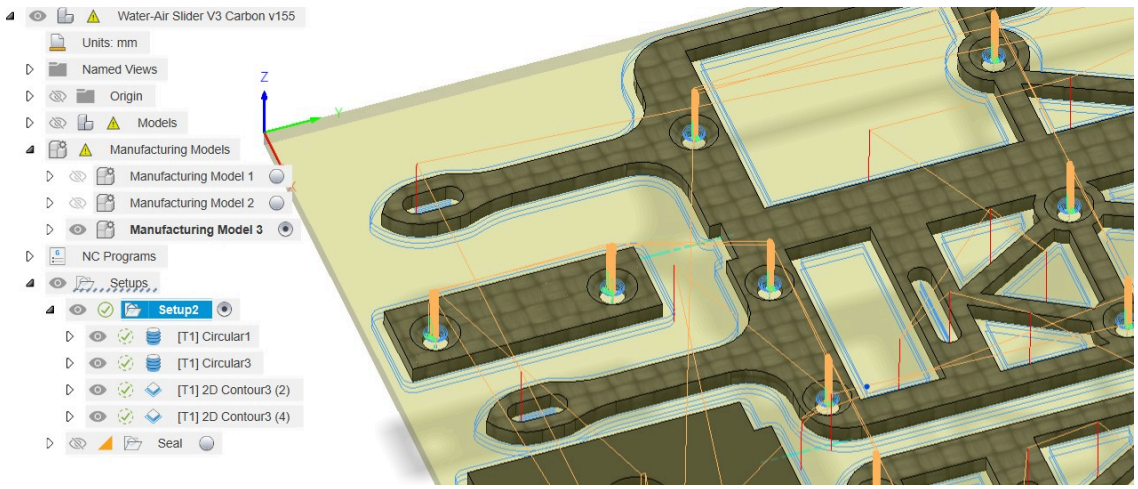


Figure 18.1: Manufacturing model in Fusion360

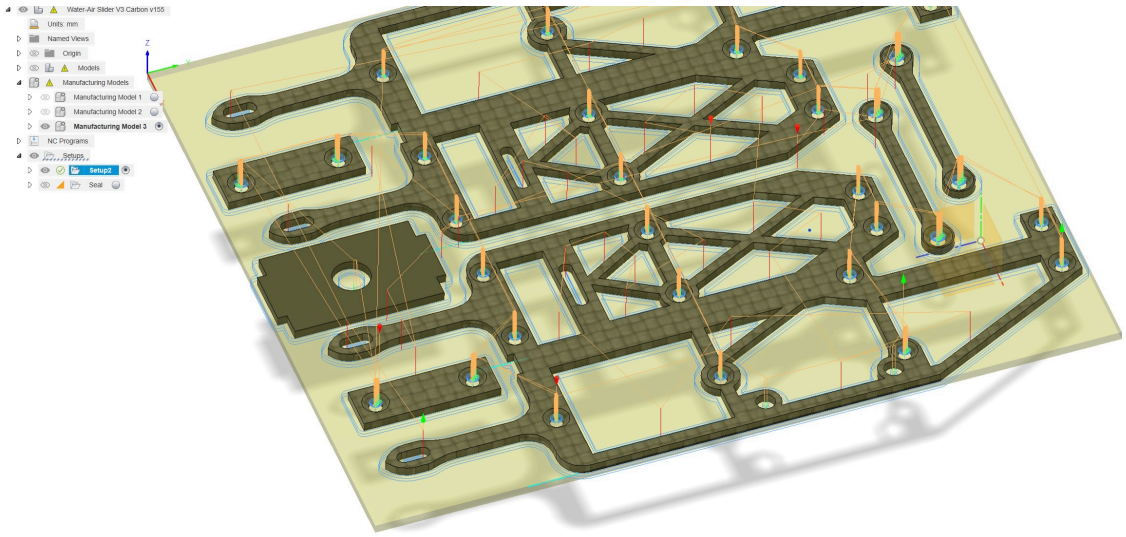


Figure 18.2: Setting up on the machine

18.3 Material-specific notes

Material	Thickness	Notes
Woven carbon	1.5 mm (varies 1.5–1.6)	Body plates, top and bottom identical except chamfers
Aluminium	5 mm	Crank and linkage. Linkage faced down from 5 to 4 mm. Use a 2 mm endmill, flood the part
PEEK	5 mm	Gears. Cutting underwater gives good results. Keep advance speed low enough to cut teeth cleanly

! MIT equivalent

The linked guide is for the EPFL machine. Confirm which machine the class will actually use at MIT and either re-shoot this guide or replace the link.

19. Design notes

Why the robot is built the way it is. Not needed at the bench — useful for your report, and for anyone changing the design.

19.1 Why an FOC ESC

Several ESC solutions were explored for the FAAV concept:

- **Regular drone ESCs.** Hobbyking 12A and EP Aer11 were tested. Both start **losing synchronisation at low speed**, which is a problem for water propulsion — precisely the regime the robot spends most of its time in.
- **FOC ESCs** tend to be more complex and larger. The **G431-ESC1** board is favourable and had already been explored in the lab.

That tradeoff — accepting a larger, more complex driver to get usable low-speed torque control — is the reason Module 3 spends a whole module on the driver.

19.2 Speed ceiling and sensor lag

Maximum speed went from **500 to 1700 rad/s**, enough to drive the wings at 10 Hz, through a combination of sensor and MCU changes documented in Module 4. The remaining constraint is sensor lag: above 500–600 rad/s, uncompensated lag shows up as high current draw and noise rather than as a hard speed limit.

Compensation is currently hand-tuned every 100 rad/s with a fitted function used on the robot. Automating this — ramping through the speeds while minimising phase current — is open work.

19.3 Gearbox corrosion

By default the gearbox on the FAAV4 is not waterproof at all, and it is normal for it to fill completely with water once submerged. This was considered acceptable: over more than a year of repeated use in water, including in Lake Geneva, a gearbox was opened and showed no sign of corrosion.

Proper lubrication — a mix of **CorrosionX and white lithium grease** - help with corrosion problems. The practical procedure is in Module 10.

19.4 Housing material tradeoff

Foam	Floatability	Absorption	Impact behaviour
Extruded polystyrene	Best, low density	Zero	Brittle; little help damping landings
Closed-cell polyethylene	Decent, relatively low density	Light	Compliant; usable as protection

A multi-material housing has never been built but would likely be better than either.

20. Changelog

Corrections and additions published during the semester, newest first.

If you are working from a printed copy, check this page before each module. Changes that affect a step you may already have completed are marked in **bold**.

> For maintainers
>
> One row per change: date, page, what changed, and whether teams need to redo anything. Do not edit a step silently — a dated entry is what lets a student at the bench trust the rest of the page.

20.1 fall-2026

Date	Page	Change	Action needed
—	—	Manual published for the Fall 2026 cohort, derived from *FAAV Manufacturing Guide* v4.0	—